CS336 作业 2（系统）：系统与并行性

版本 26.1.3

CS336 Staff

2026 年春季

目录

CS336 作业 2（系统）：系统与并行性 **1**
1 作业概述 . . . 1
你将实现的内容 . . . 1
代码结构 . . . 2
如何提交 . . . 2
2 性能分析与基准测试 . . . 2
2.1 性能分析 . . . 2
3 单 GPU 内存 . . . 8
3.1 Autograd 残留张量 . . . 8
3.2 激活检查点 . . . 10
4 GPU 内核 . . . 12
4.1 使用 FlashAttention-2 优化 Attention . . . 12
4.2 JIT 编译 Attention 基准测试 . . . 12
5 分布式数据并行训练 . . . 23
5.1 PyTorch 中的单节点分布式通信 . . . 23
5.2 分布式数据并行训练的朴素实现 . . . 25
5.3 改进最小 DDP 实现 . . . 26
6 优化器状态分片 . . . 27
7 全分片数据并行 . . . 28
8 分析并行策略 . . . 29
8.1 通信原语 . . . 29
8.2 分析数据并行 . . . 30
8.3 分析全分片数据并行 . . . 31
8.4 分析张量并行 . . . 31
8.5 2D 并行（FSDP + TP） . . . 32
9 排行榜 . . . 33
参考文献 . . . 34

CS336 作业 2（系统）：系统与并行性

版本 26.1.3
CS336 Staff
2026 年春季

1 作业概述

在本次作业中，你将获得一些实践经验：一方面提升单 GPU 训练速度，另一方面将训练扩展到多 GPU。

你将实现的内容

1. 基准测试与性能分析框架
2. 激活检查点
3. Flash Attention 2 Triton 内核

4. 分布式数据并行训练
5. 优化器状态分片
6. 全分片数据并行训练

代码结构

本作业代码和本说明文档可在 GitHub 上获得:

`github.com/stanford-cs336/assignment2-systems`

请使用 Git 克隆该仓库。如果有任何更新，我们会通知你，你可以使用 `git pull` 获取最新版本。

1. `cs336-basics/`: 在本作业中，你将对我们在作业 1 中构建的一些组件进行性能分析。此文件夹包含作业 1 的教师解答代码，因此你会在这里看到 `cs336-basics/pyproject.toml` 和 `cs336-basics/cs336_basics` 包。如果你想使用自己的模型实现，可以修改基础目录中的 `pyproject.toml` 文件，使其指向你自己的包。
2. `/`: `cs336-systems` 基础目录。我们创建了一个名为 `cs336_systems` 的空模块。请注意，这里没有代码，因此你应当可以从零开始做任何你想做的事情。
3. `tests/*.py`: 该目录包含你必须通过的所有测试。这些测试会调用 `tests/adapters.py` 中定义的 hooks。你将实现这些 adapters，以便把你的代码连接到测试中。编写更多测试和/或修改测试代码可能有助于调试你的代码，但你的实现应当通过原始提供的测试套件。
4. `README.md`: 此文件包含关于预期目录结构的更多细节，以及一些设置环境的基本说明。

如何提交

你需要向 Gradescope 提交以下文件:

- `writeup.pdf`: 回答所有书面问题。请对你的回答进行排版。
- `code.zip`: 包含你编写的所有代码。

运行 `test_and_make_submission.sh` 中的脚本以创建 `code.zip` 文件。

2 性能分析与基准测试

在作业的第一部分，我们将探索如何优化 Transformer 语言模型的性能，以尽可能高效地使用 GPU。我们将对模型进行性能分析，理解它在前向传播和反向传播期间把时间和内存花在了哪里；随后，我们将使用自定义 GPU 内核优化自注意力操作，使其比常规 PyTorch 能达到的速度更快。在本作业后续部分，我们将利用多 GPU，并理解如何在一个集群上训练模型。

2.1 性能分析

在实现任何优化之前，首先对程序进行性能分析、理解其资源消耗位置（例如时间和内存）是很有帮助的。否则，我们可能会优化模型中并不占用显著时间或内存的部分，从而看不到可测量的端到端改进。

我们将实现三条性能评估路径:

1. 使用 Python 标准库进行简单的端到端基准测试，计时前向传播和反向传播。
2. 使用 NVIDIA Nsight Systems 工具进行计算性能分析，以理解时间如何分布在 CPU 和 GPU 上的各类操作之间。
3. 内存性能分析。

2.1.1 设置 - 导入你的 basics Transformer 模型 首先，请确保你能够加载上一作业中的模型。在上一作业中，我们把模型组织为一个 Python 包，以便后续能够方便地导入。我们已经在 `./cs336-basics` 文件夹中加入了模型的参考实现，并在 `pyproject.toml` 文件中指向了它。像往常一样调用 `uv run [command]` 时，`uv` 会自动定位这个本地 `cs336-basics` 包。如果你想使用自己的模型实现，可以修改 `pyproject.toml` 文件，让它指向你自己的包。

你可以使用以下方式测试是否能够导入模型:

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
      Built cs336-systems @ file:///path/to/systems/dir
      Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr 7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

此时，作业 1 中的相关模块应当已经可用（例如，对于 `model.py`，你可以用 `import cs336_basics.model` 导入它）。

2.1.2 模型规模 在整个作业中，我们将对模型进行基准测试和性能分析，以更好地理解其性能。为了感受规模变化会带来什么影响，我们将使用并引用以下模型配置。除排行榜部分外，所有模型都使用词表大小 10,000 和 batch size 4，并改变上下文长度。本作业（以及后续作业）会要求你在表格和图中展示大量结果。我们强烈建议你用代码自动构建 writeup 中的表格，因为在 LaTeX 或 Typst 中手动格式化表格会非常繁琐。参见 `pandas.DataFrame.to_latex()` 和 `pandas.DataFrame.to_typst()`，或者编写你自己的函数，从你偏好的表格表示中生成它们。

Size	d_model	d_ff	num_layers	num_heads
small	768	3072	12	12
medium	1024	4096	24	16
large	1280	5120	36	20
xl	2560	10240	32	32
10B	4608	12288	50	36

表 1：不同模型规模的规格。这些大多基于 GPT-2 配置。

除非另有说明，请使用上下文长度 512。

2.1.3 端到端基准测试 现在我们将实现一个简单的性能评估脚本。我们将测试模型的许多变体（改变精度、替换层等），因此让你的脚本通过命令行参数启用这些变体会很有价值，因为这样之后运行起来会更容易。

首先，让我们对模型做最简单的性能分析：计时前向传播、反向传播和优化器步骤。由于我们只测量速度和内存，因此使用随机权重和随机数据是可以的。

性能测量是微妙的：一些常见陷阱会导致我们测不到真正想测的内容。对于 GPU 代码的基准测试，一个注意事项是 CUDA 调用是异步的。当你调用一个 CUDA kernel，例如调用 `torch.matmul` 时，PyTorch 函数调用会在不等待矩阵乘法完成的情况下把控制权返回给你的代码。这样一来，CPU 可以继续向前运行并调度新的操作，而 GPU 则完成矩阵乘法，这是一项重要的性能优势。另一方面，这意味着天真地测量 `torch.matmul` 调用返回需要多久，并不能告诉我们 GPU 实际运行矩阵乘法需要多久。在 PyTorch 中，我们可以调用 `torch.cuda.synchronize()`，等待所有已调度的 GPU kernels 完成，从而更准确地测量 CUDA kernel 的运行时间。此操作中的同步指的是同步 CPU 运行时和 GPU 运行时。记住这一点后，让我们编写基本的性能分析基础设施。

Problem (`benchmarking_script`)：基准测试脚本（4 分）

(a) 编写一个脚本，对模型中的前向传播、反向传播和优化器步骤进行基本的端到端基准测试。具体而言，你的脚本应支持以下功能：

- 给定超参数（例如层数），初始化一个模型。
- 生成一个随机数据 batch。
- 先运行 `w` 个 warm-up 步骤（在开始计时之前），然后计时执行 `n` 个步骤（根据一个参数，可以是仅前向传播、前向传播和反向传播，或者前向传播和反向传播加优化器步骤）。对于计时，你可以使用 Python 的 `timeit` 模块（例如使用 `timeit` 函数，或使用 `timeit.default_timer()`，它会给出系统最高分辨率的时钟，因此比 `time.time()` 更适合作为基准测试默认值）。
- 每一步之后调用 `torch.cuda.synchronize()`。

Deliverable：一个脚本，它会用给定超参数初始化一个 basics Transformer 模型，创建一个随机数据 batch，并计时仅前向传播、前向加反向传播，以及包含优化器步骤的完整训练步骤。

(b) 对 Section 2.1.2 中描述的模型尺寸计时前向、反向和优化器步骤。使用 5 个 warmup 步骤，并在 10 个测量步骤上计算计时的平均值和标准差。一次前向传播需要多长时间？反向传播呢？你是否看到测量之间存在高变异性，还是标准差很小？

Deliverable：用 1-2 句话给出你的计时结果。

(c) 基准测试的一个注意事项是没有执行 warm-up 步骤。重复你的分析，但不使用 warm-up 步骤。这会如何影响你的结果？你认为为什么会发生这种情况？也请尝试使用 1 或 2 个 warm-up 步骤运行脚本。为什么结果可能仍然不同？

Deliverable：2-3 句话的回答。

2.1.4 Nsight Systems 分析器 端到端基准测试并不能告诉我们模型在前向和反向传播期间把时间和内存花在了哪里，因此也无法暴露具体的优化机会。为了知道程序在每个组件（例如函数）中花了多少时间，我们可以使用 profiler。执行 profiler 会通过在函数开始和结束运行时插入 guard 来检测代码，因此能够给出函数级别的详细执行统计信息（例如调用次数、平均耗时、累计耗时等）。

标准 Python profiler（例如 CProfile）无法分析 CUDA kernels，因为这些 kernels 在 GPU 上异步执行。幸运的是，NVIDIA 提供了一个可以通过 CLI `nsys` 使用的 profiler。我们建议你从包管理器获取最新版本，或使用下载页上的安装程序。在这一部分作业中，你将使用 `nsys` 分析 Transformer 模型的运行时。

使用 `nsys` 很直接：在上一节的 Python 脚本前加上 `nsys profile` 运行。例如，你可以对脚本 `benchmark.py` 运行一个基本 profile：

```
$ uv run nsys profile -- python benchmark.py
```

随后，你可以在本地机器上用 NVIDIA Nsight Systems 桌面应用查看该 profile。在 profile 的 `CUDA API` 行中选择某个特定 CUDA API 调用（在 CPU 上）时，会在 `CUDA HW` 行中高亮所有对应的 kernel 执行（在 GPU 上）。

更全面的一次 profiling 运行可能如下所示：

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx
 ↪  --pytorch=functions-trace,autograd-shapes-nvtx --cudabacktrace=all --python-backtrace=cuda
 ↪  --gpu-metrics-devices=0 -- python benchmark.py
```



图 1: 图 1：详细的 Nsight Systems trace

图 1：详细的 Nsight Systems trace。

图中文字翻译：`GPU Metrics` → GPU 指标；`Activity` → 活跃度；`Memory` → 内存；`GPU Active` → GPU 活跃；`Compute in Flight` → 正在执行的计算；`SMs Active` → SM 活跃；`SM Instructions` → SM 指令；`SM Warp Occupancy` → SM Warp 占用率；`DRAM Bandwidth` → DRAM 带宽；`PCIe Rx/Tx Throughput` → PCIe 接收/发送吞吐量；`Process` → 进程；`CUDA HW` → CUDA 硬件；`Memory Usage` → 内存使用量；`NVTX` → NVTX；`PyTorch` → PyTorch；`Threads` → 线程。

在这个例子中，`--trace` 指定要记录哪些 API，`--pytorch` 会在 module 调用和 autograd 期间插入 `nvtx` 标签，`--cudabacktrace` 和 `--python-backtrace` 会提供更好的 backtrace，以便理解代码中的某个 kernel 是从哪里被调用的，而 `--gpu-metrics-devices` 指定要测量哪块 GPU 的利用率。

给一次运行加入 profiling 并不是免费的，它总体上会拖慢运行。通常只启用你在某次运行中想看的功能是值得的。具体来说，当不需要 traceback 时，你可能想移除 `--cudabacktrace=all` 和 `--python-backtrace=cuda`，因为它们有非常大的开销。

我们鼓励你尝试 `nsys profile` 的各种命令行选项，以了解它能做什么。你还可以用 NVTX ranges 标注你的代码，它们会在 profile 的 NVTX 行中显示为块，捕获所有 CUDA API 调用及相关 kernel 执行。特别地，你应当使用 NVTX ranges 来忽略基准测试脚本中的 warm-up 步骤（通过在 profile 中对 `nvtx` 标签应用 `--nvtx-capture` 过滤器）。你也可以隔离出哪些 kernels 对应于模型的前向和反向传播，甚至可以通过如下方式标注实现来隔离自注意力层不同部分对应的 kernels：

```
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
```



图 2: 图 1：详细的 Nsight Systems trace

```
    ... # Q, K, V, mask
):
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection

    return ...
```

你可以在基准测试脚本中通过以下方式把原始实现替换为带标注的版本：

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

最后值得注意的是，`torch.compile` 可能让时间和资源难以归因到代码的具体部分。你可能需要用 `torch.compile` 和 `nvtx` 标注包装并剥离代码中的不同部分，才能正确地把时间和资源使用归因到源代码的各个部分。

Problem (`nsys_profile`)：Nsight Systems 性能分析（5 分）

使用 `nsys` profile 你的前向传播、反向传播和优化器步骤。请选择 Table 1 中的两个模型尺寸，以及三个大于 128 的 2 的幂上下文长度，其中最大可用尺寸应当是你能放入内存的最长上下文长度。选择你认为最有趣的组合。对于每个 profile，回答以下问题：

(a) 前向传播总共花了多少时间？它是否与我们之前用 Python 标准库测量的结果一致？

Deliverable：1-2 句话的回答。

(b) 在前向传播中，哪个 CUDA kernel 占用的累计 GPU 时间最多？在模型的一次前向传播中，该 kernel 被调用了多少次？当你同时进行前向和反向传播时，耗时最多的是否是同一个 kernel？（提示：查看 `Stats System View` 下的 `CUDA GPU Kernel Summary`，并使用 NVTX ranges 过滤，以识别模型哪些部分对应哪些 kernels。）

Deliverable：1-2 句话的回答。

(c) 尽管绝大多数 FLOPs 发生在矩阵乘法中，你会注意到其他一些 kernels 仍然占用了非平凡比例的整体运行时间。除了矩阵乘法之外，在前向传播中你还看到哪些 kernels 占用了非平凡的 CUDA 运行时间?

Deliverable：1-2 句话的回答。

(d) Profile 运行一个完整训练步骤，即使用你实现的 AdamW：前向传播、计算 loss 并运行反向传播，最后执行优化器步骤，就像训练时会做的那样。与推理（仅前向传播）相比，花在矩阵乘法上的时间比例如何？其他 kernels 呢?

Deliverable：1-2 句话的回答。

(e) 比较模型自注意力层前向传播中 softmax 操作与矩阵乘法操作的运行时间。运行时间差异与 FLOPs 差异相比如何?

Deliverable：1-2 句话的回答。

2.1.5 混合精度 到目前为止，本作业一直使用 FP32 精度运行：所有模型参数和激活都具有 `torch.float32` datatype。然而，现代 NVIDIA GPU 包含专门的 GPU 核心（Tensor Cores），可用于加速低精度矩阵乘法。例如，NVIDIA B200 规格表称其 FP32 最大吞吐量为 80 TFLOPS，而 FP16（半精度浮点数）或 BF16（bfloat16）的最大吞吐量则显著更高，高达 2500 TFLOPS。因此，使用低精度 datatype 应当有助于加速训练和推理。

然而，天真地把模型转换为低精度格式可能会导致模型精度下降。例如，在实践中许多梯度值常常小到无法用 FP16 表示，因此在用 FP16 精度天真训练时会变成零。为了解决这个问题，在 FP16 训练中通常使用 loss scaling：简单地把 loss 乘以一个缩放因子，从而增大梯度幅值，使其不会 flush to zero。此外，FP16 的动态范围比 FP32 更低，这可能导致溢出并表现为 NaN loss。完整 bfloat16 训练通常更稳定（因为 BF16 与 FP32 具有相同的动态范围），但与 FP32 相比，仍然可能影响最终模型性能。

为了利用低精度 datatype 的加速, 常见做法是使用 mixed-precision training。在 PyTorch 中, 这是通过 `torch.autocast` context manager 实现的。在这种情况下，某些操作（例如矩阵乘法）以低精度 datatype 执行，而其他需要 FP32 完整动态范围的操作（例如累加和归约）保持不变。例如，以下代码会在前向传播期间自动识别哪些操作应当以低精度执行，并将这些操作转换为指定的数据类型：

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.bfloat16
x : torch.Tensor = ... # input data

with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

如上所述，即使被累加的张量本身已经被 downcast，让累加以更高精度进行通常也是一个好主意。下面的练习将帮助你建立关于为什么如此的直觉。

Problem (`mixed_precision_accumulation`)：混合精度累加（1 分）

运行以下代码，并评论结果的准确性。

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)

s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01,dtype=torch.float16)
    s += x.type(torch.float32)
print(s)
```

Deliverable：2-3 句话的回答。

现在，我们将先把混合精度应用到一个 toy model 上以建立直觉，然后再应用到基准测试脚本中。

Problem (benchmarking_mixed_precision)：混合精度基准测试（2 分）

(a) 考虑以下模型：

```
class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x
```

假设我们正在 GPU 上训练该模型，并且模型参数最初是 FP32。我们希望使用 FP16 的 autocasting mixed precision。以下各项的数据类型是什么：

- autocast context 中的模型参数？
- 第一个 feed-forward 层（`ToyModel.fc1`）的输出？
- layer norm（`ToyModel.ln`）的输出？
- 模型预测的 logits？
- loss？
- 模型的 gradients？

Deliverable：列出上述每个组件的数据类型。

(b) 你应当已经看到，FP16 mixed precision autocasting 对 layer normalization 层和 feed-forward 层的处理方式不同。layer normalization 的哪些部分对混合精度敏感？如果我们使用 BF16 而不是 FP16，是否仍然需要不同地处理 layer normalization？为什么？

Deliverable：2-3 句话的回答。

(c) 修改你的基准测试脚本，使其能够可选地使用 BF16 mixed precision 运行模型。对 Section 2.1.2 中描述的每个语言模型尺寸，分别在使用和不使用 mixed precision 的情况下计时前向和反向传播。比较使用 full precision 与 mixed precision 的结果，并评论随着模型尺寸变化出现的趋势。你可能会发现 `nullcontext` no-op context manager 很有用。

Deliverable：2-3 句话的回答，包含你的计时和评论。

2.1.6 内存性能分析 到目前为止，我们一直关注计算性能。现在我们将把注意力转向内存，这是语言模型训练和推理中的另一个主要资源。PyTorch 还提供了强大的内存 profiler，可以跟踪随时间变化的分配情况。

要使用内存 profiler，你可以按如下方式修改基准测试脚本：

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

这会输出一个文件 `memory_snapshot.pickle`，你可以将其加载到以下在线工具中：`pytorch.org/memory_viz`。该工具会让你看到整体内存使用时间线，以及每个单独的分配：包括其大小，以及指向其来源代码的 stack trace。要使用该工具，你应当在浏览器中打开上面的链接，然后把 pickle 文件拖放到页面上。

现在，你将使用 PyTorch profiler 来分析模型的内存使用。

Problem (memory_profiling)：内存性能分析（4 分）

Profile Table 1 中 `xl` 模型在上下文长度 128 和 2048 下的完整训练步骤，包括前向传播、反向传播和优化器步骤。

(a) 给你的 profiling 脚本添加一个选项，使模型通过内存 profiler 运行。

复用你之前的一些基础设施可能会有帮助（例如激活 mixed precision、加载特定模型尺寸等）。然后运行脚本，获取 `xl` 模型在仅推理（只做前向传播）或完整训练步骤时的内存 profile。你的内存时间线是什么样的？你能根据看到的峰值判断当前正在运行哪个阶段吗？

Deliverable：来自 `memory_viz` 工具的两张 `xl` 模型 `Active memory timeline` 图片：一张用于前向传播，一张用于运行完整训练步骤（前向和反向传播，然后优化器步骤），以及 2-3 句话的回答。

(b) 在做前向传播时，每个上下文长度的峰值内存使用是多少？完整训练步骤又是多少？

Deliverable：一个表格，每个上下文长度包含两个数字。

(c) 找到使用 mixed precision 时 `xl` 模型在前向传播和完整训练步骤中的峰值内存使用。mixed precision 是否显著影响内存使用？

Deliverable：2-3 句话的回答。

(d) 考虑 `xl` 模型。给定我们的参考超参数，Transformer residual stream 中一个 activation tensor 在单精度下的大小是多少？请用 MiB 给出该大小（即字节数除以 `1024^2`）。

Deliverable：1-2 句话，包含你的推导。

(e) 现在仔细查看 `pytorch.org/memory_viz` 中 `xl` 模型执行前向传播时的内存 snapshot 对应的 `Active Memory Timeline`。当你降低 `Detail` 级别时，该工具会隐藏对应级别以下的最小分配（例如，把 `Detail` 设为 10% 时只显示最大的 10% 分配）。显示的最大分配有多大？查看 stack trace，你能判断这些分配来自哪里吗？

Deliverable：1-2 句话的回答。

(f) Nsight Systems 也有用于内存 profiling 的 flags。你可以将这些 flags 与之前的 Nsight flags 结合起来，以理解模型生命周期不同步骤中发生了哪些分配。使用 PyTorch 提供的 NVTX labels 来确定模型中单个 `TransformerBlock` 为反向传播保存了多少内存（这些张量通常称为 residuals）。记录最大的 5 个贡献操作，以及它们占整体内存的百分比。

在反向传播期间，所有这些张量都会被释放，但新的 gradient tensors 也会同时被产生。基于你的 profiles（展示前向传播期间分配了多少内存，以及反向传播中每个 `TransformerBlock` 的内存使用如何变化），计算一个 `TransformerBlock` 产生的 gradient tensors 占用多少内存。结果是否符合你的预期？

Deliverable：Nsight Systems 截图，以及 1-2 段回答。

3 单 GPU 内存

本作业后续部分将探索把张量分片到多个 GPU 上的技巧，但也有一些技巧即使在单 GPU 训练中也能应用。其中最常见的是 gradient checkpointing（也称 activation checkpointing）。

3.1 Autograd 残留张量

回忆一下，为了通过模型执行反向传播，我们需要保存前向传播中产生的 activations。虽然对于某些操作来说这显然是必要的，但默认情况下，保存的张量数量会比你预想的多得多。为反向传播保存的张量称为 “residuals”，或简称 “saved tensors”。

让我们对网络中保存了什么建立一些理解。从看似简单的 RMSNorm 函数开始（为简化起见使用纯 FP32），我们添加一些 hooks，在 autograd 保存或取回张量时触发。

```
import torch
from torch import nn

x = torch.randn((4, 512, 2560), requires_grad=True)


class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps
```

```
    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x


def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t


def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

ln = RMSNorm(x.shape[-1])

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

输出显示有数量令人担忧的张量被写出，其中有几个是完整 activation 大小!

```
$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None

Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
```

3.1.1 算子融合 在这个例子中，显然所使用操作的粒度太高了。我们希望有一个单一 op，它接受 RMSNorm 权重和 activation，并输出结果；同时也希望该操作在反向传播中作为一个整体单元。这正是 kernel fusion 的动机之一。由于 RMSNorm 的行为相当良好，我们甚至可以用 `torch.compile` 自动融合它。

```
...
ln = torch.compile(RMSNorm(x.shape[-1]))

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

新的输出显著更好:

```
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
```

```
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
```

我们只需要为反向传播保存一个完整大小的 activation tensor，也就是 RMSNorm 函数的输入。还要注意，加载顺序不再是保存顺序的反向，并且每个 residual 不再有 `grad_fn` 依赖：PyTorch 正把整个 RMSNorm 视为一个单一函数。

3.2 激活检查点

尽管融合无疑很有用，但它在节省内存方面只能帮我们到一定程度。例如，让我们融合一个 `xl` 尺寸的单个 `TransformerBlock`。

```python
import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock

# num_layers for this model is 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(d_model=d_model, d_ff=d_ff, num_heads=num_heads,
positional_encoder=RotaryEmbedding(dim=d_model // num_heads, context_length=context_length))

# Fuse as much torch.compile will allow
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)

...
# Now logs the number of bytes saved
total_size_bytes = 0
def pack_hook(t):
    if isinstance(t, torch.nn.Parameter): # Skip logging parameters to avoid double counting
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

...

# Run forward pass, saving for backward
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)

print(f"Total size of saved tensors in single TransformerBlock: {total_size_bytes /
(1024**2):.2f} MiB")
```

该脚本显示了我们为反向传播保存了多少内存：

```
...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB
```

每一层 3.6 GiB。如果对所有层都这样做，仅为反向传播保存的 activations 就会达到 114 GiB！注意力操作的 residuals 中存在非平凡的浪费，我们会在 Section 4 中修复这一点；但即使修复后，内存使用仍会随 batch size、sequence length 和 embedding size 线性增长。

3.2.1 重计算 与其保留我们生成的每个张量，不如只保存结果的周期性 checkpoints，并重新计算中间值。PyTorch 提供了一个接口，可以用简单方式处理这一点。`torch.utils.checkpoint.checkpoint` 接受一个函数及其参数。它随后通过以下方式修改传入函数的行为：

1. 在前向传播中：
    1. 保存该函数的输入值。
    2. 抑制在前向传播中保存张量。
2. 在反向传播中：
    1. 在前面加入一个重计算步骤：从先前保存的输入重新计算前向传播，并保存反向传播所需的值。
    2. 执行反向传播，随后所有张量都可以被释放。

在连续运行 4 个 transformer blocks 的简单例子中，我们看到内存会如预期般累加。

```
...
def four_blocks(x):
    x = block(x)
     x = block(x)
     x = block(x)
     x = block(x)
     return x


with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)

print(f"Total size of saved tensors in four TransformerBlocks: {total_size_bytes /
(1024**2):.2f} MiB")
```

```
Total size of saved tensors in four TransformerBlocks: 14605.25 MiB
```

但我们可以如下使用 gradient checkpointing:

```
from torch.utils.checkpoint import checkpoint
def two_blocks(x):
    x = block(x)
    x = block(x)
    return x


def four_blocks_checkpoint(x):
    # checkpoint throws out all the saved tensors until the backward pass
    # when getting to the checkpointed block in the backward pass,
    # it reruns a forward pass to produce the saved tensors,
    # then completes normal backward pass.
    x = checkpoint(two_blocks, x, use_reentrant=False)
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)

print(f"Total size of saved tensors in four TransformerBlocks with checkpointing:
{total_size_bytes / (1024**2):.2f} MiB")
```

```
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB
```

请注意，这并没有消除内存使用。相反，它把内存使用分解成两类：一类是在每次 checkpoint 调用入口处为准备重计算而保存的较长期存储（checkpoint 本身），另一类是在 checkpointed block 内部的重计算 pass 中产生的短期内存，用于支持通过它进行反向传播。由于我们的主要考虑是峰值内存使用，因此我们希望平衡已保存 checkpoints 的内存成本与物化完整 block residuals 的内存成本。使用的 checkpoints 越多，单个 block 内物化的内存越小，但 checkpoints 本身需要的内存越多。

在上面的例子中，很明显重计算 residuals 的内存占主导地位（部分原因是我们的 checkpoints 很小），因此把平衡更多地转向 checkpointed memory 会是有益的。也就是说，我们希望减小 checkpointed block 的作用范围。

较大或较小的 checkpointed blocks 不会影响重计算的计算成本，但我们可以通过使用 recursive checkpointing 继续以计算换取更低内存需求，也就是在 checkpoint 调用内部嵌套 checkpoint 调用。

Problem (`gradient_checkpointing`)：内存最优 Gradient Checkpointing（4 分）

考虑一个由 `N` 个相同 blocks 顺序堆叠而成的 Transformer。如果没有任何 checkpointing，所有 `N` 个 blocks 的 residuals 都会同时保持存活，从而产生 `O(N)` 的峰值 activation memory。我们可以自由地用 `checkpoint` 包装前向传播的任意子集，包括在彼此内部嵌套 checkpoint 调用。

(a) 忽略计算成本时，哪种 checkpointing 策略能最小化峰值 activation memory? 描述你将如何安排 checkpoint 调用（代码草图即可），并给出该策略作为 N 的函数时的渐近峰值 activation memory 和计算量。假设单个 block 保存的 residuals 主导所有每 checkpoint bookkeeping。

Deliverable：用 3-5 句话描述策略及其渐近峰值内存，并附一个简短代码草图。

(b) 考虑上面的 `xl` 模型配置：batch size 4，sequence length 2048。如果你只有时间/计算预算运行一步重计算（也就是说不能嵌套 checkpoint 调用），最佳的 checkpointing 策略是什么，以降低峰值内存? Profile 你的运行峰值内存以验证假设。比较下一个更小和更大的 checkpointing block sizes 的峰值内存以确认。

Deliverable：3-5 句话描述你的推理，以及你策略的实测峰值内存。

4 GPU 内核

4.1 使用 FlashAttention-2 优化 Attention

4.1.1 PyTorch Attention 基准测试 你的 profiling 很可能表明，在注意力层中，无论是内存还是计算都有优化机会。在高层次上，attention 操作由一次矩阵乘法、随后 softmax、再随后另一次矩阵乘法组成：

```
Attention(Q, K, V) = softmax(mask(QK^T / sqrt(d_k)))V    (1)
```

朴素 attention 实现需要为每个 batch/head 元素保存形状为 `seq_len × seq_len` 的 attention score matrices；当 sequence length 很长时，它们会变得非常大，导致任何长输入或长输出任务出现 out-of-memory errors。我们将按照 FlashAttention-2 论文实现一个 attention kernel，它按 tiles 计算 attention，并避免显式物化 `seq_len × seq_len` attention score matrices，从而能够扩展到更长的 sequence lengths。

Problem (`pytorch_attention`): PyTorch Attention 基准测试（2 分）

(a) 在不同规模下 benchmark 你的 attention 实现。编写一个脚本，要求：

(b) 将 batch size 固定为 8，并且不要使用 multihead attention（即移除 head 维度）。

(ii) 对 head embedding 维度 `d_model` 的 `[16, 32, 64, 128]` 和 sequence length 的 `[256, 1024, 4096, 8192, 16384]` 进行笛卡尔积遍历。

(iii) 为合适的尺寸创建随机输入 `Q, K, V`。

(iv) 使用输入计时 100 次 attention 前向传播。

(v) 测量在 backward pass 开始前使用了多少内存，并计时 100 次 backward passes。

(vi) 确保进行 warm up，并在每次 forward/backward pass 后调用 `torch.cuda.synchronize()`。

根据你的 GPU，其中一些配置预计会 out of memory。报告这些配置对应的计时结果（或 out-of-memory errors）。你在哪个尺寸遇到 out-of-memory errors? 对你发现的最小 out-of-memory 配置之一进行 attention 内存使用核算（可以使用作业 1 中 Transformer 内存使用的公式）。随 sequence length 变化，为 backward 保存的内存如何变化? 你会怎么做以消除此内存成本?

Deliverable：一个包含计时结果的表格、你的内存使用计算，以及 1-2 段回答。

4.2 JIT 编译 Attention 基准测试

自 2.0 版本以来，PyTorch 还提供了一个强大的 just-in-time compiler，它会自动尝试对 PyTorch 函数应用多种优化：入门介绍见 `https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html`。特别地，它会通过动态分析你的 computation graph 来尝试自动生成融合的 Triton kernels。使用 PyTorch compiler 的接口非常简单。例如，如果我们想把它应用到模型中的某一层，可以使用：

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

现在，`compiled_layer` 在功能上就像 `layer` 一样（例如具有其前向和反向传播）。我们也可以使用 `torch.compile(model)` 编译整个 PyTorch 模型，甚至可以编译一个调用 PyTorch 操作的 Python 函数。

Problem (`torch_compile`): Torch Compile（2 分）

(a) 扩展你的 attention 基准测试脚本，加入 PyTorch attention 实现的 compiled version，并在与 `pytorch_attention` 问题相同的配置下比较它与未编译版本的性能。

Deliverable：一个表格，对比你的 compiled attention module 与 `pytorch_attention` 问题中未编译版本的前向和反向传播计时。

(b) 现在，在端到端基准测试脚本中编译整个 Transformer 模型。前向传播性能如何变化? combined forward and backward passes 以及 optimizer steps 的性能又如何变化?

Deliverable：一个表格，对比 vanilla Transformer 模型与 compiled Transformer 模型。

鉴于我们已经看到关于 sequence length 的 scaling behaviors，需要显著改进才能处理大 sequence。即使使用 `torch.compile`，当前实现在长 sequence length 下仍然存在非常糟糕的内存访问模式。为此，我们将编写 FlashAttention-2 的 Triton 实现，这样能够更精细地控制如何访问内存以及何时计算什么。

4.2.1 示例 - Weighted Sum 为了介绍你需要了解的 Triton 以及它如何与 PyTorch 互操作，我们将从一个 "weighted sum" 操作的示例 kernel 入手。关于快速上手 Triton 的更多资源，请参见 Triton tutorials。我们注意到这些 tutorials 没有使用新的、方便的 block pointer 抽象；我们将在下面介绍它。

给定输入矩阵 `X`，我们将把其中的元素乘以一个按列的权重向量 `w`，然后对每一行求和，从而得到 `X` 和 `w` 的矩阵-向量乘积。我们先走一遍该操作的前向传播，然后为反向传播编写 Triton kernel。

Forward pass

我们 kernel 的 forward pass 就是如下广播 inner product：

```
def weighted_sum(x, weight):
    # Here, assume that x has n-dim shape [..., D], and weight has 1D shape [D]
    return (weight * x).sum(axis=-1)
```

编写 Triton kernel 时，我们将让每个 program instance（可能并行运行）计算 `x` 的一个 row tile 的 weighted sum，并把对应的标量输出写入 output tensor。在 Triton 中，program instance 是一个执行相同程序的线程块，这些线程块可以在 GPU 上并行运行。我们不把 tensors 作为参数传入，而是传入指向其第一个元素的 pointers，以及每个 tensor 的 strides；这些 strides 告诉我们如何沿各个轴移动。

我们可以使用 strides 加载一个 tensor，它对应于当前运行 instance 正在求和的 `x` 的 row tile，并使用 program ID 划分工作（即 instance `i` 会处理 `x` 的第 `i` 个 row tile）。在这个简单例子中，Triton forward pass 与 PyTorch forward pass 的主要区别在于需要进行 pointer arithmetic 和显式 loads/stores。我们将使用 `tl.make_block_ptr` 提供的 block pointer 抽象来大幅简化 pointer arithmetic，尽管这意味着我们需要做一些设置来准备 block pointers。

图 2：weighted sum kernel 示例中的 tiling 和 block pointers advance（Section 4.2.1）。

图中文字翻译：`ROWS` → 行；`ROWS_TILE` → 行 tile；`D` → D；`D_TILE` → D tile；`.advance((ROWS_TILE, 0))` 与 `.advance((0, D_TILE))` 保持原文。

参见 Figure 2，了解 tiling 以及 block pointers 如何 advance 的示意图。上面的 weighted sum 函数如下所示：

```
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr, # Input pointers
    output_ptr, # Output pointer
    x_stride_row, x_stride_dim, # Strides tell us how to move one element in each axis of a tensor
    weight_stride_dim, # Likely 1
    output_stride_row, # Likely 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr, # Tile shapes must be known at
    ↪ compile time
):
    # Each instance will compute the weighted sum of a tile of rows of x.
    # `tl.program_id` gives us a way to check which thread block we're running in
    row_tile_idx = tl.program_id(0)

    # Block pointers give us a way to select from an ND region of memory
    # and move our selection around.
    # The block pointer must know:
    # - The pointer to the first element of the tensor
    # - The overall shape of the tensor to handle out-of-bounds access
    # - The strides of each dimension to use the memory layout properly
    # - The ND coordinates of the starting block, i.e., "offsets"
    # - The block shape to load/store at a time
    # - The order of the dimensions in memory from major to minor
    #    axes (= np.argsort(strides)) for optimizations, needed for
    #    TMA support on >=Hopper
```

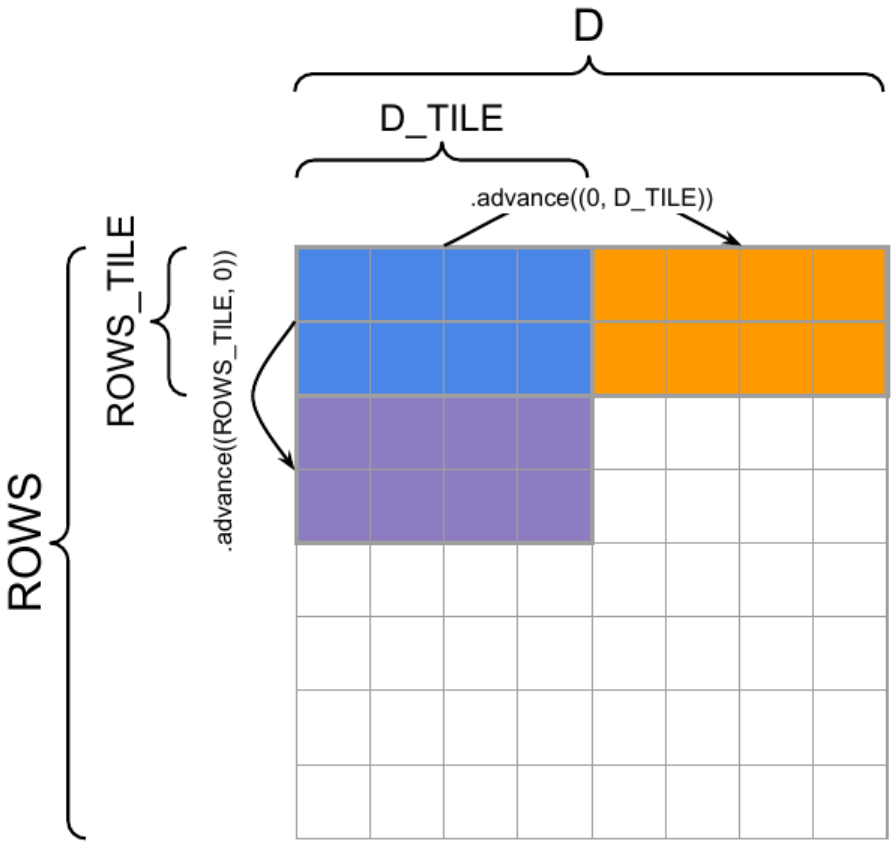


图 3: 图 2：weighted sum kernel 示例中的 tiling 和 block pointers advance（Section 4.2.1）

```
    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,),
        strides=(x_stride_row, x_stride_dim),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,),
        strides=(weight_stride_dim,),
        offsets=(0,),
        block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    output_block_ptr = tl.make_block_ptr(
        output_ptr,
        shape=(NUM_ROWS,),
        strides=(output_stride_row,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    # Initialize a buffer to write to
    output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        # Load the current block pointer
        # Since ROWS_TILE_SIZE might not divide NUM_ROWS, and D_TILE_SIZE might not divide D,
        # we need boundary checks for both dimensions
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") #
↪   (ROWS_TILE_SIZE, D_TILE_SIZE)
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero") #
↪   (D_TILE_SIZE,)

        # Compute the weighted sum of the row.
        output += tl.sum(row * weight[None, :], axis=1)

        # Move the pointers to the next tile.
        # These are (rows, columns) coordinate deltas
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE)) # Move by D_TILE_SIZE in the last
↪   dimension
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,)) # Move by D_TILE_SIZE

    # Write output to the output block pointer (a single scalar per row).
    # Since ROWS_TILE_SIZE might not divide NUM_ROWS, we need boundary checks
    tl.store(output_block_ptr, output, boundary_check=(0,))
```

现在，让我们把这个 kernel 包装到一个 PyTorch Autograd function 中，使其能够与 PyTorch 互操作（即以 Tensors 作为输入、输出一个 Tensor，并在之后的 backward pass 中也能与 autograd engine 一起工作）：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # Cache x and weight to be used in the backward pass, when we
        # only receive the gradient wrt. the output tensor, and
        # need to compute the gradients wrt. x and weight.
```

```
        D, output_dims = x.shape[-1], x.shape[:-1]

        # Reshape input tensor to 2D
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")

        ctx.save_for_backward(x, weight)

        assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
        assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
        assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"

        ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16 # Roughly 16 loops through the
↪   embedding dimension
        ctx.ROWS_TILE_SIZE = 16 # Each thread processes 16 batch elements at a time
        ctx.input_shape = input_shape

       # Need to initialize empty result tensor. Note that these elements are not necessarily 0!
        y = torch.empty(output_dims, device=x.device)

        # Launch our kernel with n instances in our 1D grid.
        n_rows = y.numel()
        weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
            x, weight,
            y,
            x.stride(0), x.stride(1),
            weight.stride(0),
            y.stride(0),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
        )

        return y.view(input_shape[:-1])
```

请注意，当我们用 `weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)]` 调用 Triton kernel 时，我们通过传入 tuple `(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)` 定义了所谓的线程块 launch grid。随后，我们可以在 kernel 中用 `tl.program_id(0)` 访问线程块索引。

Backward pass

由于我们定义了自己的 kernel，因此还需要编写自己的 backward function。

在 forward pass 中，我们给定 layer 的输入，并需要计算其输出。在 backward pass 中，回忆一下，我们会得到目标函数关于输出的 gradients，并需要计算关于每个输入的 gradient。在我们的例子中，该操作的输入是矩阵 `x : R^{n x h}` 和权重向量 `w : R^h`。为简洁起见，把该操作称为 `f(x, w)`，其值域为 `R^n`。随后，假设给定 `∇_{f(x,w)} L`，即 loss `L` 关于 layer 输出的 gradient，则可以应用多元链式法则，得到关于 `x` 和 `w` 的 gradients 的如下表达式：

```
(∇_x L)_{ij} = sum_{k=1}^n ∂f(x,w)_k / ∂x_{ij} (∇_{f(x,w)} L)_k = w_j * (∇_{f(x,w)} L)_i     (2)
(∇_w L)_j = sum_{i=1}^n ∂f(x,w)_i / ∂w_j (∇_{f(x,w)} L)_i = sum_{i=1}^n x_{ij} * (∇_{f(x,w)}
↪  L)_i     (3)
```

这给出了计算 backward pass 的简单公式。为了获得关于 `x` 的 backward step，我们应用 Equation 2，并取 `w` 与 `∇_{f(x,w)}` 的 outer product。为了计算关于 `w` 的 backward step（即 `(∇_w L)_j`），我们必须把 input gradient 乘以对应的 output row。

我们的 backward pass kernel 会先定义所有 block pointers，然后计算 `∇_x L`：

```
@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr, # Input
    grad_output_ptr, # Grad input
    grad_x_ptr, partial_grad_weight_ptr,       # Grad outputs
    stride_xr, stride_xd,
    stride_wd,
    stride_gr,
    stride_gxr, stride_gxd,
```

```
    stride_gwb, stride_gwd,
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)

    # Inputs
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS,), strides=(stride_gr,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,), strides=(stride_wd,),
        offsets=(0,), block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D,), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )

   for i in range(tl.cdiv(D, D_TILE_SIZE)):
       grad_output = tl.load(grad_output_block_ptr, boundary_check=(0,), padding_option="zero")
↪  #
(ROWS_TILE_SIZE,)

       # Outer product for grad_x
       weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")   #
↪  (D_TILE_SIZE,)
       grad_x_row = grad_output[:, None] * weight[None, :]
       tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))

       # Reduce as many rows as possible for the grad_weight result
       row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero") #
↪  (ROWS_TILE_SIZE, D_TILE_SIZE)
```

```
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,)) # Never out
    ↪ of bounds for dim
0


        # Move the pointers to the next tile along D
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
        partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
        grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))
```

计算 gradient `∇_x` 很简单，我们把结果写入 output tensor 的对应 tile。然而，计算 `∇_w` 更具挑战性。每个 kernel instance 负责 `x` 的一个 row tile，但现在我们需要跨 `x` 的 rows 求和。我们不在 backward pass 中直接做这个求和，而是假设 `partial_grad_weight_ptr` 包含一个 `n_row_tiles × H` 矩阵，其中第一个维度只在来自 `x` 的一个 row tile 内部被 reduce。我们在当前 row tile 内部 reduce，然后写入该 tensor。在 kernel 外部，我们使用 `torch.sum` reduce `∇_w`，对每个 row tile 的结果求和。[1]

`autograd.Function` 的最后一部分相对简单：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (defined earlier)

        @staticmethod
        def backward(ctx, grad_out):
            x, weight = ctx.saved_tensors
            ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE   # These don't
    ↪ have to be the same
            n_rows, D = x.shape

            # Our strategy is for each thread block to first write to a partial buffer,
            # then we reduce over this buffer to get the final gradient.
            partial_grad_weight = torch.empty((triton.cdiv(n_rows, ROWS_TILE_SIZE), D),
    ↪ device=x.device, dtype=x.dtype)
            grad_x = torch.empty_like(x)

            weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
                x, weight,
                grad_out,
                grad_x, partial_grad_weight,
                x.stride(0), x.stride(1),
                weight.stride(0),
                grad_out.stride(0),
                grad_x.stride(0), grad_x.stride(1),
                partial_grad_weight.stride(0), partial_grad_weight.stride(1),
                NUM_ROWS=n_rows, D=D,
                ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
            )
            grad_weight = partial_grad_weight.sum(axis=0)
            return grad_x, grad_weight
```

最后，我们现在可以获得一个函数，它的工作方式很像 `torch.nn.functional` 中实现的那些函数：

```
f_weightedsum = WeightedSumFunc.apply
```

现在，在两个 PyTorch tensors `x` 和 `w` 上调用 `f_weightedsum` 会得到如下 tensor：

```
tensor([ 90.8563, -93.6815, -80.8884, ..., 103.4840, -21.4634, -24.0192],
       device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

请注意附着在 tensor 上的 `grad_fn`：这表明当该 tensor 出现在 computation graph 中时，PyTorch 知道在 backward pass 中应该调用什么。这就完成了 weighted sum 操作的 Triton 实现。

[1] 当然，我们也可以为此编写自己的 kernel。

4.2.2 FlashAttention-2 前向传播 你将用一个显著改进的 Triton 实现替换你的 PyTorch attention 实现，该实现遵循 FlashAttention-2 [T. Dao, 2023]。FlashAttention-2 使用一些技巧按 tiles 计算 forward pass，从而实现高效内存访问模式，并避免在 global memory 上物化完整 attention matrix。

在进入本节之前，我们强烈建议你至少阅读原始 FlashAttention 论文 [T. Dao et al., 2022]，它会让你对 FlashAttention 实现高效 attention 的核心技术建立直觉：跨 tiles 以 online 方式计算 softmax（这是 [M. Milakov et al., 2018] 提出的技术）。我们还建议你查看 H. He [4]，以获得更多关于 GPU 实际如何执行 PyTorch 代码的直觉。

理解 vanilla attention 中的低效

回忆一下，attention 的 forward pass（暂时忽略 masking）可以写为：

```
S = QK^T / sqrt(d)                                (4)
P_ij = softmax_j(S)_ij                            (5)
O = PV                                             (6)
```

标准 backward pass 为：

```
dV = P^T dO                                        (7)
dP = dO V^T                                        (8)
dS_i = dsoftmax(dP_i) = (diag(P_i) - P_i P_i^T)dP_i    (9)
dQ = dS K / sqrt(d)                                (10)
dK = dS^T Q / sqrt(d)                              (11)
```

正如我们所见，backward pass 依赖于 forward pass 中的一些非常大的 activations。例如，Equation 7 中计算 `dV` 需要 `P`，它是形状为 `(batch_size, n_heads, seq_len, seq_len)` 的 attention scores：该 activation matrix 的大小与 sequence length 的平方成正比，这解释了我们上面在大 sequence length 下 benchmark attention 时遇到的内存问题。在 vanilla attention 的 forward 和 backward pass 中，我们都要支付显著的 memory IO 成本，用于在 on-chip SRAM 和 GPU HBM 之间传输 `P` 和其他大型 activations。标准实现中会发生多个这样的传输：例如，标准 backward pass 实现会在 Equation 7 和 Equation 9 的计算中都从 HBM 读取 `P`。

FlashAttention 的主要目标是避免把 attention matrix 写入 HBM 或从 HBM 读出，从而降低 IO 和峰值内存成本。我们通过三种技术实现这一点：tiling、recomputation 和 operator fusion。

Tiling

为了避免从 HBM 读写 attention matrix，我们在无法访问完整输入的情况下计算 softmax reduction。具体而言，我们重构 attention 计算，把输入拆分为 tiles，并对输入 tiles 进行多次 pass，从而增量地执行 softmax reduction。

Recomputation

我们避免在 HBM 中存储形状为 `(batch_size, n_heads, seq_len, seq_len)` 的大型中间 attention matrices。相反，我们将在 HBM 中保存某些 "activation checkpoints"，然后在 backward pass 期间重计算部分 forward pass，以获得计算 gradients 所需的其他 activations。FlashAttention-2 还会存储 attention scores 的 logsumexp，即 `L`，它将用于简化 backward pass 计算。`L` 的表达式为：

```
L_i = log(sum_j exp(S_ij))                         (12)
```

在最终 kernel 中，我们会以 online 方式计算它，但最终结果应当相同。结合 tiling 与 recomputation，我们的 memory IO 和峰值使用不再依赖于 `sequence_length^2`，因此可以使用更大的 sequence lengths。

Operator fusion

最后，我们通过在一个 kernel 中执行所有操作来避免对 attention matrix 和其他中间 activations 的重复 memory IO，这称为 operator fusion 或 kernel fusion。我们将为 forward pass 编写一个单一 Triton kernel，它用有限的 HBM 与 SRAM 之间数据传输执行 attention 中涉及的所有操作。Operator fusion 部分由 recomputation 启用，因为我们可以避免通常为了把每个中间 activation 存入 HBM 而支付的 memory IO。

关于这些技术的更多直觉，请参见 FlashAttention 论文 [T. Dao et al., 2022; T. Dao, 2023]。

带重计算的 backward pass

使用 `L`，我们可以执行合适的重计算，并高效计算 backward pass。在开始 backward pass 之前，我们在 global memory 中预计算值 `D = rowsum(O ∘ dO)`（其中 ∘ 是 element-wise multiplication）。它等于 `rowsum(P ∘ dP)`，因为 `P dP^T = P(dO V^T)^T = (PV)dO^T = O dO^T`，并且对于任意矩阵 `A` 和 `B`，`rowsum(A ∘ B) = diag(AB^T)`。有了 `L` 和 `D` vectors，backward pass 可以不用 softmax 计算。完整的 backward pass 计算现在为：

```
S = QK^T / sqrt(d)                                 (13)
P_ij = exp(S_ij - L_i)                             (14)
dV = P^T dO                                        (15)
```

```
dP = dO V^T                                (16)
dS_ij = P_ij(dP_ij - D_i)                  (17)
dQ = dS K / sqrt(d)                        (18)
dK = dS^T Q / sqrt(d)                      (19)
```

我们可以看到，这一系列操作不要求我们在 forward pass 期间把 attention scores `P` 存储到 HBM 中：我们会从 activations `Q`、`K` 和 `L` 通过 Equation 13 和 Equation 14 重计算它们。

FlashAttention forward pass 的细节

现在我们已经对 FlashAttention-2 使用的技术有了高层理解，接下来深入你将实现的 FA2 forward pass kernel 的细节。为了避免从 HBM 读取和写入 attention matrix，我们希望使用 tiling，即独立于其他 tiles 计算输出的每个 tile。这要求我们能够计算 `P` 的 tiles，理想情况下沿两个维度（queries 和 keys）都进行 tiling。

然而，当我们对 `S` 应用 softmax 时，需要 reduce `S` 的整行来计算 softmax denominator，这意味着不能直接按 tiles 计算 `P`。FlashAttention-2 使用 online softmax 解决了这个问题。在下面的文字中，我们用下标 index `i` 表示当前 query tile，用上标 index `(j)` 表示当前 key tile。query 维度上的 tile 大小为 `B_q`，key 维度上的 tile 大小为 `B_k`。我们不会沿 hidden dimension `d` 做 tiling。

我们还保留一些 row-wise running values，`m_i^{(j)} ∈ R^{B_q}` 和 `l_i^{(j)} ∈ R^{B_q}`。row-wise `m_i^{(j)}` 是 running maximum，用于以数值稳定方式计算 softmax（回忆作业 1 中 softmax 实现中的这个技巧）。每当出现新的 row-wise tile of `S`（即 `j` 增加时），我们会更新 `m_i^{(j)}`。使用 running maximum，我们可以计算未归一化的 softmax values（numerators）为 `Ptilde_i^{(j)} = exp(S_ij - m_i^{(j)})`。`l_i^{(j)}` 是 softmax denominator 的 running proxy，并会用未归一化 softmax values 更新为 `l_i^{(j)} = exp(m_i^{(j-1)} - m_i^{(j)}) * l_i^{(j-1)} + rowsum(Ptilde_i^{(j)})`。当最终写入输出时，我们需要使用 `l_i^{(T_k)}` 完成归一化，它是处理完所有 key tiles 后 `l_i^{(j)}` 的最终值。Algorithm 1 展示了 GPU 上的 forward pass。

Algorithm 1: FlashAttention-2 forward pass

1. Require: `Q ∈ R^{N_q x d}`, `K, V ∈ R^{N_k x d}`, tile sizes `B_q`, `B_k`
2. Split `Q` into `T_q = ceil(N_q / B_q)` tiles `Q_1, ..., Q_{T_q}` of size `B_q x d`
3. Split `K, V` into `T_k = ceil(N_k / B_k)` tiles `K^(1), ..., K^(T_k)` and `V^(1), ..., V^(T_k)` of size `B_k x d`
4. for `i = 1, ..., T_q` do
5. Load `Q_i` from global memory
6. Initialize `O_i^(0) = 0 ∈ R^{B_q x d}`, `l_i^(0) = 0 ∈ R^{B_q}`, `m_i^(0) = -∞ ∈ R^{B_q}`
7. for `j = 1, ..., T_k` do
8. Load `K^(j), V^(j)` from global memory
9. Compute tile of pre-softmax attention scores `S_i^(j) = Q_i (K^(j))^T / sqrt(d) ∈ R^{B_q x B_k}`
10. Compute `m_i^(j) = max(m_i^(j-1), rowmax(S_i^(j))) ∈ R^{B_q}`
11. Compute `Ptilde_i^(j) = exp(S_i^(j) - m_i^(j)) ∈ R^{B_q x B_k}`
12. Compute `l_i^(j) = exp(m_i^(j-1) - m_i^(j)) l_i^(j-1) + rowsum(Ptilde_i^(j)) ∈ R^{B_q}`
13. Compute `O_i^(j) = diag(exp(m_i^(j-1) - m_i^(j)))O_i^(j-1) + Ptilde_i^(j)V^(j)`
14. end for
15. Compute `O_i = diag(l_i^(T_k))^{-1} O_i^(T_k)`
16. Compute `L_i = m_i^(T_k) + log(l_i^(T_k))`
17. Write `O_i` to global memory as the `i`-th tile of `O`.
18. Write `L_i` to global memory as the `i`-th tile of `L`.
19. end for
20. Return the output `O` and the logsumexp `L`.

在进入 Triton forward pass 实现之前，我们在这里汇总一些编写 Triton kernels 的通用技巧。

Triton Tips and Tricks

- 你可以在 Triton 中使用 `tl.device_print` 打印语句进行调试：`https://triton-lang.org/main/python-api/generated/triton.language.device_print.html`。有一个设置 `TRITON_INTERPRET=1` 可以在 CPU 上运行 Triton interpreter，不过我们发现它有 bug。
- 定义 block pointers 时，确保它们具有正确的 offsets，并确保 block offsets 乘以合适的 tile sizes。
- 线程块 launch grid 设置如下：

```
kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...)
```

这出现在 `torch.autograd.Function` 子类的方法中，正如 weighted sum 示例中看到的那样。- 使用 `tl.dot` 执行矩阵乘法。- 要 advance 一个 block pointer，请使用 `*_block_ptr = *_block_ptr.advance(...)`。

Problem (`flash_forward`): FlashAttention-2 Forward Pass（15 分）

(a) 编写一个纯 PyTorch（无 Triton）的 `autograd.Function`，实现 FlashAttention-2 forward pass。这会比常规 PyTorch 实现慢得多，但有助于你调试 Triton kernel。

你的实现应当以 `Q`、`K` 和 `V` 以及一个 flag `is_causal` 作为输入，并产生输出 `O` 和 logsumexp 值 `L`。在该任务中可以忽略 `is_causal` flag。`autograd.Function` 的 `forward` 随后应当保存 `L, Q, K, V, O` 供 backward pass 使用，并返回 `O`。记住，`autograd.Function` 的 `forward` 方法的实现总是以 context 作为第一个参数。任何 `autograd.Function` 类都需要实现一个 `backward` 方法，但目前你可以让它直接 `raise NotImplementedError`。如果你需要比较对象，可以在 PyTorch 中实现 Equation 4 到 Equation 6 以及 Equation 12，并比较输出。

接口为 `def forward(ctx, Q, K, V, is_causal=False)`。自行决定 tile sizes，但确保它们至少为 `16 × 16`。我们总是会用维度为 2 的幂且至少为 16 的输入测试你的代码，因此你无需担心 out-of-bounds accesses。

Deliverable：一个 `torch.autograd.Function` 子类，实现 FlashAttention-2 forward pass。为了测试你的代码，请实现 `[adapters.get_flashattention_autograd_function_pytorch]`。然后运行 `uv run pytest -k test_flash_forward_pass_pytorch`，并确保实现通过测试。

(b) 按照 Algorithm 1 为 FlashAttention-2 的 forward pass 编写 Triton kernel。随后，编写另一个 `torch.autograd.Function` 子类，让其在 forward pass 中调用这个（fused）kernel，而不是在 PyTorch 中计算结果。几个问题特定的提示：

- 为了调试，我们建议你把每个 Triton 操作的结果与 part (a) 中编写的 tiled PyTorch 实现进行比较。
- 你的 launch grid 应设置为 `(T_q, batch_size)`，这意味着每个 Triton program instance 只会从单个 batch index 加载元素，并只读/写 `Q`、`O` 和 `L` 的单个 query tile。
- kernel 应当只有一个 loop，它会遍历 key tiles `1 <= j <= T_k`。
- 在 loop 末尾 advance block pointers。
- 使用以下函数声明（借助我们给你的 block pointer，你应当能够推断其余 pointers 的设置）：

```
@triton.jit
def flash_fwd_kernel(
    Q_ptr, K_ptr, V_ptr,
    O_ptr, L_ptr,
    stride_qb, stride_qq, stride_qd,
    stride_kb, stride_kk, stride_kd,
    stride_vb, stride_vk, stride_vd,
    stride_ob, stride_oq, stride_od,
    stride_lb, stride_lq,
    N_QUERIES, N_KEYS,
    scale,
    D: tl.constexpr,
    Q_TILE_SIZE: tl.constexpr,
    K_TILE_SIZE: tl.constexpr,
):
    # Program indices
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)

    # Offset each pointer with the corresponding batch index
    # multiplied with the batch stride for each tensor
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )

    ...
```

其中 `scale` 是 `1 / sqrt(d)`，`Q_TILE_SIZE` 和 `K_TILE_SIZE` 分别为 `B_q` 和 `B_k`。之后你可以调优这些值。

这些额外指南可能有助于你避免精度问题：

- on-chip buffers（`O_i, l, m`）应具有 `dtype tl.float32`。如果你正在累加到 output buffer，请使用 `acc` 参数（`acc = tl.dot(..., acc=acc)`）。

- 在与 V^(j) 相乘之前，将 Ptilde_i^(j) cast 到 V^(j) 的 dtype，并在写入 global memory 之前将 O_i cast 到合适的 dtype。cast 使用 tensor.to 完成。你可以用 tensor.dtype 获取 tensor 的 dtype，用 *_block_ptr.type.element_ty 获取 block pointer/pointer 的 dtype。

Deliverable：一个 torch.autograd.Function 子类，使用你的 Triton kernel 实现 FlashAttention-2 forward pass。实现 [adapters.get_flash_autograd_function_triton]。随后运行 uv run pytest -k test_flash_forward_pass_triton，并确保实现通过测试。

(c) 为你的 autograd.Function 实现添加最后一个参数 flag，用于 causal masking。这应当是一个 boolean flag，当设置为 True 时，启用用于 causal masking 的 index comparison。你的 Triton kernel 应具有对应的额外参数 is_causal: tl.constexpr（这是必需的类型标注）。在 Triton 中，构造合适的 query 和 key index vectors，并比较它们以形成大小为 B_q × B_k 的 square mask。对于被 mask 掉的元素，将常数值 -1e6 加到 attention score matrix S_i^(j) 的对应元素上。确保使用 ctx.is_causal = is_causal 保存 mask flag 供 backward 使用。

Deliverable：为你的 torch.autograd.Function 子类添加一个 flag，使用 Triton kernel 实现带 causal masking 的 FlashAttention-2 forward pass。确保该 flag 是可选的，并默认值为 False，这样之前的测试仍能通过。

使用重计算实现 backward pass

请注意，与 Equation 7 到 Equation 11 中的标准 backward pass 不同，我们可以使用 recomputation 来避免 Equation 13 到 Equation 19 所示 backward pass 中的 softmax 操作。这意味着可以使用一个简单 kernel 计算 backward pass，并且不需要 online 技巧。因此，在这一部分，你可以通过对一个常规 PyTorch 函数调用 torch.compile 来实现 backward（不是 Triton）。

Problem (flash_backward): FlashAttention-2 Backward Pass（5 分）

使用 PyTorch（非 Triton）和 torch.compile 为你的 FlashAttention-2 autograd.Function 实现 backward pass。你的实现应当以 Q, K, V, O, dO, L tensors 作为输入，并返回 dQ, dK, dV。记得计算并使用 D vector。你可以按照 Equation 13 到 Equation 19 的计算进行。

Deliverable：为了测试你的实现，运行 uv run pytest -k test_flash_backward。

现在，让我们比较你（部分）Triton 实现的 FlashAttention-2 与常规 Attention 的 PyTorch 实现的性能。

Problem (flash_benchmarking): FlashAttention-2 基准测试（5 分）

(a) 编写一个使用 triton.testing.do_bench 的基准测试脚本，比较你（部分）Triton 实现的 FlashAttention-2 forward 和 backward passes 与常规 PyTorch 实现（即不使用 FlashAttention）的性能。

具体而言，你需要报告一个表格，其中包括 forward、backward 和端到端 forward-backward pass 的 latency，对你的 Triton 实现和 PyTorch 实现都要报告。在开始 benchmark 之前随机生成任何必要输入，并在单个 B200 上运行 benchmark。始终使用 batch size 1 和 causal masking。遍历以下笛卡尔积：sequence length 为从 128 到 65536 的若干 2 的幂，embedding dimension size 为从 16 到 128 的若干 2 的幂，precision 为 torch.bfloat16 和 torch.float32。你很可能需要根据输入尺寸调整 tile sizes。

Deliverable：使用上述设置比较你的 FlashAttention-2 实现与 PyTorch 实现的结果表格，并报告 forward、backward 和端到端 latencies。

4.2.3 OPTIONAL: Triton backward pass 如果你希望获得更多 Triton 练习和/或拥有更快的 leaderboard submission，我们在下面提供了 tiled FlashAttention-2 backward pass，你可以在 Triton 中实现。Algorithm 2 展示了应当在 Triton 中实现的 FlashAttention-2 backward pass。这里的一个关键技巧是计算 P 两次，一次用于 dQ，另一次用于 dK 和 dV。这让我们跳过线程块之间的同步，从而可以避免缓慢的 atomics。

Algorithm 2: Tiled FlashAttention-2 backward pass

1. Require: Q, O, dO ∈ R^{N_q x d}, K, V ∈ R^{N_k x d}, L ∈ R^{N_q}, tile sizes B_q, B_k
2. Compute D = rowsum(O ∘ dO) ∈ R^{N_q}
3. Split Q, O, dO into T_q = ceil(N_q / B_q) tiles Q_1, ..., Q_{T_q}, O_1, ..., O_{T_q}, dO_1, ..., dO_{T_q}, each of size B_q × d
4. Split K, V into T_k = ceil(N_k / B_k) tiles K^(1), ..., K^(T_k) and V^(1), ..., V^(T_k), each of size B_k × d
5. Split L, D into T_q tiles L_1, ..., L_{T_q} and D_1, ..., D_{T_q}, each of size B_q
6. for j = 1, ..., T_k do
7. Load K^(j), V^(j) from global memory
8. Initialize dK_0^(j) = dV_0^(j) = 0 ∈ R^{B_k × d}
9. for i = 1, ..., T_q do
10. Load Q_i, dO_i from global memory
11. Compute tile of attention scores S_i^(j) = Q_i (K^(j))^T / sqrt(d) ∈ R^{B_q × B_k}
12. Compute attention probabilities P_i^(j) = exp(S_i^(j) - L_i) ∈ R^{B_q × B_k}

13. Compute dV_i^(j) = dV_{i-1}^(j) + (P_i^(j))^T dO_i ∈ R^{B_k × d}
14. Compute dP_i^(j) = dO_i (V^(j))^T ∈ R^{B_q × B_k}
15. Compute dS_i^(j) = P_i^(j) ∘ (dP_i^(j) - D_i) ∈ R^{B_q × B_k}
16. Compute dK_i^(j) = dK_{i-1}^(j) + (dS_i^(j))^T Q_i / sqrt(d) ∈ R^{B_k × d}.
17. end for
18. Write dK_{T_q}^(j) and dV_{T_q}^(j) to global memory as the j-th tiles of dK and dV.
19. end for
20. for i = 1, ..., T_q do
21. Load Q_i, dO_i from global memory
22. Initialize dQ_i^(0) = 0 ∈ R^{B_q × d}
23. for j = 1, ..., T_k do
24. Load K^(j), V^(j) from global memory
25. Compute tile of attention scores S_i^(j) = Q_i (K^(j))^T / sqrt(d) ∈ R^{B_q × B_k}
26. Compute attention probabilities P_i^(j) = exp(S_i^(j) - L_i) ∈ R^{B_q × B_k}
27. Compute dP_i^(j) = dO_i (V^(j))^T ∈ R^{B_q × B_k}
28. Compute dS_i^(j) = P_i^(j) ∘ (dP_i^(j) - D_i) ∈ R^{B_q × B_k}
29. Compute dQ_i^(j) = dQ_i^(j-1) + dS_i^(j) K^(j) / sqrt(d) ∈ R^{B_q × d}
30. end for
31. Write dQ_i^(T_k) to global memory as the i-th tile of dQ.
32. end for
33. Return dQ, dK, dV.

5 分布式数据并行训练

在作业的下一部分，我们将探索使用多 GPU 训练语言模型的方法，重点关注数据并行。我们将从 PyTorch 中分布式通信的入门开始。然后，我们会研究分布式数据并行训练的朴素实现，并随后实现和 benchmark 多种提升通信效率的方法。

5.1 PyTorch 中的单节点分布式通信

首先来看 PyTorch 中的一个简单分布式应用，其目标是生成四个随机整数 tensors 并计算它们的和。

在下面的分布式情形中，我们会 spawn 四个 worker processes，每个 worker 都生成一个随机整数 tensor。为了跨 worker processes 对这些 tensors 求和，我们将调用 all-reduce collective communication operation，它会把每个 process 上的原始 data tensor 替换为 all-reduced 结果（即求和结果）。

现在来看一些代码。

```python
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

运行上面的脚本后，我们得到下面的输出。正如预期，每个 worker process 最初持有不同的 data tensors。经过 all-reduce 操作后，该操作会跨所有 worker processes 对 tensors 求和，`data` 会在每个 worker process 上被原地修改为 all-reduced 结果。[2]

[2] 如果你多次运行该脚本，会注意到打印输出的顺序不是确定性的。由于该应用在分布式设置中运行，我们无法控制命令运行的精确顺序；我们唯一能保证的是，当 all-reduce 操作完成后，各个 processes 会持有 bitwise identical 的结果 tensors。

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

现在再更仔细地回看上面的脚本。命令 `mp.spawn` 会 spawn `nprocs` 个进程，这些进程运行 `fn` 并传入给定的 `args`。此外，函数 `fn` 会以 `fn(rank, *args)` 的形式被调用，其中 `rank` 是 worker process 的索引（取值在 `0` 到 `nprocs-1` 之间）。因此，我们的 `distributed_demo` 函数必须把这个整数 `rank` 作为其第一个位置参数。此外，我们传入 `world_size`，它指 worker processes 的总数。

每个 worker process 都属于一个 process group，该 process group 通过 `dist.init_process_group` 初始化。process group 表示多个 worker processes，它们会通过一个共享 master 协调并通信。master 由其 IP 地址和端口定义，并运行 rank 0 的进程。像 all-reduce 这样的 collective communication operations 会作用于 process group 中的每个 process。

在这个例子中，我们使用 `"gloo"` backend 初始化 process group，但也有其他 backend 可用。特别地，`"nccl"` backend 会使用 NVIDIA NCCL collective communications library，对于 CUDA tensors 通常性能更高。然而，NCCL 只能在带 GPU 的机器上使用，而 Gloo 可以在仅 CPU 的机器上运行。分布式 GPU 训练应当始终使用 NCCL；只有在没有 GPU 可用的本地开发场景中才使用 Gloo。我们在这个例子中使用 Gloo，是因为它允许在仅 CPU 的机器上进行本地执行和开发。

运行 multi-GPU jobs 时，请确保不同 ranks 使用不同 GPUs。一种方法是在 `setup` 函数中调用 `torch.cuda.set_device(rank)`，这样 `tensor.to("cuda")` 就会自动把 tensor 移动到指定设备。或者，你也可以显式创建每个 rank 的 device string（例如 `device = f"cuda:{rank}"`），然后在任何数据移动中使用这个 device string 作为目标设备（例如 `tensor.to(f"cuda:{rank}")`）。

术语

在本作业其余部分（以及你可能在线上看到的其他资源中），你可能会在 PyTorch 分布式通信语境中遇到以下术语。虽然本作业重点关注单节点、多进程分布式训练，但这些术语有助于理解一般分布式训练。视觉表示见 Figure 3。

`node`：网络中的一台机器。

`worker`：参与分布式训练的程序实例。在本作业中，每个 worker 都有一个单独 process，因此我们会交替使用 worker、process 和 worker process。不过在实践中，一个 worker 也可能使用多个 processes（例如用于加载训练数据），因此这些术语并不总是等价。

`world size`：一个 process group 中的总 worker 数量。

`global rank`：一个整数 ID（在 `0` 到 `world_size-1` 之间），用于唯一标识 process group 中的一个 worker。例如，当 world size 为 2 时，一个 process 的 global rank 为 0（master process），另一个 process 的 rank 为 1。

`local world size`：当应用跨不同 nodes 运行时，local world size 是在给定 node 本地运行的 workers 数量。例如，如果我们有一个应用在 2 个 nodes 上各 spawn 4 个 workers，则 world size 为 8，local world size 为 4。注意，在单节点上运行时，一个 worker 的 local world size 等同于（global）world size。

`local rank`：一个整数 ID（在 `0` 到 `local_world_size-1` 之间），用于唯一标识机器上某个本地 worker 的索引。例如，如果一个应用在 2 个 nodes 上各 spawn 4 个 processes，则每个 node 都会有 local ranks 为 0、1、2 和 3 的 workers。注意，当运行单节点多进程分布式应用时，一个 process 的 local rank 等同于其 global rank。

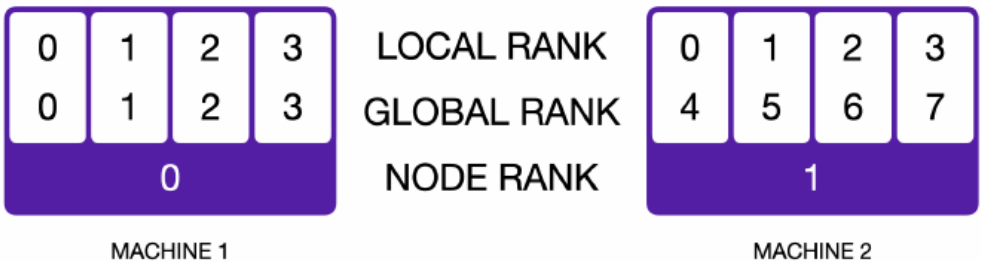


图 4: 图 3：运行在 2 个 nodes 上、world size 为 8 的分布式应用示意图

图 3：运行在 2 个 nodes 上、world size 为 8 的分布式应用示意图。每个 worker process 都由一个 global rank（从 0 到 7）和一个 local rank（从 0 到 3）标识。图取自 `https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html`。

图中文字翻译：`LOCAL RANK` → 本地 rank；`GLOBAL RANK` → 全局 rank；`NODE RANK` → 节点 rank；`MACHINE 1` → 机器 1；`MACHINE 2` → 机器 2。

5.1.1 分布式应用基准测试的最佳实践 在本作业的这一部分，你将对分布式应用进行 benchmark，以更好地理解通信带来的开销。以下是一些最佳实践：

- 尽可能在同一台机器上运行 benchmarks，以便进行受控比较。
- 在计时感兴趣的操作之前，执行若干 warm-up 步骤。这对于 NCCL communication calls 尤其重要。通常 5 次 warmup 迭代就足够了。
- 在 GPU 上 benchmark 时，调用 `torch.cuda.synchronize()` 等待 CUDA 操作完成。请注意，即使使用 `async_op=False` 调用通信操作，这也是必要的，因为该调用在操作被 queued 到 GPU 时返回（而不是通信实际完成时返回）。[3]
- 不同 ranks 上的计时可能略有不同，因此常见做法是跨 ranks 聚合测量值，以改进估计。你可能会发现 all-gather collective（特别是 `dist.all_gather_object` 函数）可用于从所有 ranks 收集结果。
- 一般来说，在 CPU 上用 Gloo 本地调试，然后根据具体问题要求，在 GPU 上用 NCCL benchmark。

在 backend 之间切换应当只需修改 `init_process_group` 调用和 tensor device casts。

Problem (`distributed_communication_single_node`)：分布式通信（单节点）（5 分）

编写一个脚本，在单节点多进程设置中 benchmark all-reduce 操作的运行时间。上面的示例代码可以作为一个合理起点。实验改变以下设置：

`all-reduce data size`：`float32` data tensors，大小为 1MB、10MB、100MB、1GB。

`Number of GPUs/processes`：2、4 或 6。

Resource requirements：最多 6 块 GPU。每次 benchmark 运行应少于 5 分钟。

Deliverable：用于比较各种设置的图和/或表，以及 2-3 句话评论你的结果，并说明你对各种因素如何相互作用的看法。

5.2 分布式数据并行训练的朴素实现

现在我们已经了解了如何在 PyTorch 中编写分布式应用的基础，让我们构建一个最小的 distributed data parallel（DDP）训练实现。

Data parallelism 会把 batches 切分到多个 devices（例如 GPUs）上，从而支持训练单个 device 无法容纳的大 batch size。例如，给定四个 devices，每个 device 最多可处理 batch size 32，则 data parallel training 可以支持 effective batch size 128。

以下是朴素执行 distributed data parallel training 的步骤。最初，每个 device 构造一个（随机初始化的）模型。我们使用 broadcast collective communication operation，把 rank 0 上的模型参数发送到所有其他 ranks。训练开始时，每个 device 都持有一份相同的模型参数和 optimizer states（例如 Adam 中累积的 gradient statistics）。

1. 给定一个包含 `n` 个样本的 batch，该 batch 会被 sharded，每个 device 接收 `n/d` 个不相交样本（其中 `d` 是用于 data parallel training 的 devices 数量）。`d` 应整除 `n`，否则某些 ranks 会做更多工作，而该 step 会被最慢的 rank 瓶颈化。
2. 每个 device 使用本地模型参数副本，在它的 `n/d` 个样本上运行 forward pass，并运行 backward pass 计算 gradients。注意，此时每个 device 持有它所接收的 `n/d` 个样本计算得到的 gradients。
3. 然后，我们使用 all-reduce collective communication operation，跨不同 devices 对 gradients 取平均，使每个 device 都持有所有 `n` 个样本的平均 gradients。
4. 接下来，每个 device 运行 optimizer step 来更新其参数副本：从 optimizer 的角度看，它只是在优化一个本地模型。由于它们从相同的初始模型和 optimizer state 出发，并在每次迭代中使用相同的平均 gradients，因此不同 devices 上的参数和 optimizer states 会保持同步。此时，我们完成了单次训练迭代，可以重复该过程。

Problem (`naive_ddp`)：朴素 DDP（5 分）

Deliverable：实现一种朴素形式的 distributed data parallel training，在 backward pass 后对单个参数 gradients 逐个 all-reduce。为了测试你的实现，请实现 `[adapters.get_ddp]` 和（可选）`[adapters.ddp_on_after_backward]`，然后运行 `uv run pytest tests/test_ddp.py`。

Problem (`naive_ddp_benchmarking`)：朴素 DDP 基准测试（3 分）

在这个朴素 DDP 实现中，parameter gradients 会在每次 backward pass 后跨 ranks 逐个 all-reduce。为了更好地理解 data parallel training 的开销，请创建一个脚本，在使用该朴素 DDP 实现训练你之前实现的语言模型时进行 benchmark。测量每个 training step 的总时间，以及用于通信 gradients 的时间比例。在单节点设置（1 node x 2 GPUs）中收集 `xl` 模型尺寸（见 Section 2.1.2）的测量结果。

[3] 详见 `https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386`。

Deliverable：你的 benchmark 设置描述，以及每种设置下测得的每次 training iteration 时间和用于通信 gradients 的时间。

5.3 改进最小 DDP 实现

我们在 Section 5.2 中看到的最小 DDP 实现有几个关键限制：

1. 它对每个 parameter tensor 都执行一次单独的 all-reduce 操作。每次通信调用都会产生开销，因此把通信调用 batch 起来以最小化这一开销可能有益。
2. 它会等待 backward pass 完成后才通信 gradients。然而，backward pass 是增量计算的。因此，当某个 parameter gradient 准备好时，可以立即通信，而不必等待其他参数的 gradients。这允许我们把 gradient 通信与 backward pass 的计算重叠起来，从而降低 distributed data parallel training 的开销。

在这一部分，我们将依次解决这些限制，并测量其对训练速度的影响。

5.3.1 减少通信调用次数 与其为每个 parameter tensor 发起一次通信调用，不如看看是否可以通过 batching all-reduce 提升性能。具体而言，我们会取出想要 all-reduce 的 gradients，将其连接成一个单一 tensor，然后跨所有 ranks 对合并后的 gradients 进行 all-reduce。使用 `torch._utils._flatten_dense_tensors` 和 `torch._utils._unflatten_dense_tensors` 可能会有帮助。

Problem (`minimal_ddp_flat_benchmarking`)：Flat Gradients 的最小 DDP 基准测试（2 分）

修改你的最小 DDP 实现，使其通信一个包含所有参数 flattened gradients 的 tensor。在之前使用的条件下（1 node x 2 GPUs，`xl` 模型尺寸见 Section 2.1.2），比较其性能与对每个 parameter tensor 发起一次 all-reduce 的最小 DDP 实现的性能。

Deliverable：在使用单次 batched all-reduce 调用的 distributed data parallel training 下，测得的每次 training iteration 时间和用于通信 gradients 的时间。用 1-2 句话比较 batching 与逐个通信 gradients 的结果。

5.3.2 将计算与单个参数 gradients 的通信重叠 虽然 batching 通信调用可能帮助降低大量小 all-reduce 操作带来的开销，但所有通信时间仍直接构成开销。为了解决这一点，我们可以利用这样一个观察：backward pass 会逐层增量计算 gradients（从 loss 开始向输入方向移动）；因此，我们可以在 parameter gradients 准备好后立即 all-reduce 它们，通过把 backward pass 计算与 gradient 通信重叠，降低 data parallel training 的开销。

我们将先实现并 benchmark 一个 distributed data parallel wrapper，它会在 backward pass 中，当单个 parameter tensors 准备好时异步 all-reduce 它们。以下指针可能有用：

`Backward hooks`：为了在 backward pass 中一个 parameter 的 gradient 被累积之后自动调用某个函数，可以使用 `register_post_accumulate_grad_hook` 函数。[4]

`Asynchronous communication`：所有 PyTorch collective communication operations 都支持 synchronous（`async_op=False`）和 asynchronous execution（`async_op=True`）。Synchronous calls 会阻塞，直到 collective operation 被 queued 到 GPU。这并不意味着 CUDA operation 已完成，因为 CUDA operations 是异步的。尽管如此，后续使用输出的函数调用会按预期行为执行。[5] 相比之下，asynchronous calls 会返回一个 distributed request handle；因此，当函数返回时，collective communication operation 不保证已经被 queued 到 GPU，更不用说完成了。为了等待该操作被 queued 到 GPU（从而其输出可用于后续操作），你可以对返回的 communication handle 调用 `handle.wait()`。

例如，以下两个示例分别用 synchronous 或 asynchronous 调用 all-reduce 一个 tensors 列表中的每个 tensor：

```
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous, block until operation is queued on the GPU.
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous, return immediately after each call and
# wait on results at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)

# ...
```

[4] 更多信息和使用示例见 `https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html`。

[5] 在高级情形中，如果你使用多个 CUDA streams，可能需要在 streams 之间显式同步，以确保输出对后续操作可用。见 `https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams`。

```
# Possibly execute other commands that don't rely on the all_reduce results
# ...

# Ensure that all-reduce calls were queued and
# therefore other operations depending on the
# all-reduce output can be queued.
for handle in handles:
    handle.wait()
handles.clear()
```

Problem (`ddp_overlap_individual_parameters`)：重叠单个参数通信的 DDP（5 分）

实现一个 Python 类来处理 distributed data parallel training。该类应当包装任意 PyTorch `nn.Module`，并负责在训练前 broadcast weights（使所有 ranks 具有相同初始参数），以及为 gradient averaging 发起通信调用。我们推荐以下 public interface：

`def __init__(self, module: torch.nn.Module)`：给定一个要并行化的已实例化 PyTorch `nn.Module`，构造一个会处理跨 ranks gradient synchronization 的 DDP container。

`def forward(self, *inputs, **kwargs)`：使用提供的位置参数和关键字参数调用被包装 module 的 `forward()` 方法。

`def finish_gradient_synchronization(self)`：调用时，等待异步通信调用在 GPU 上完成。

要使用这个类进行分布式训练，我们会传入一个 module 来包装它，然后在运行 `optimizer.step()` 之前调用 `finish_gradient_synchronization()`，以确保 optimizer step（一个依赖 gradients 的操作）可以被安全 queued：

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

Deliverable：实现一个 container class 来处理 distributed data parallel training。该类应当重叠 gradient communication 与 backward pass 的计算。为了测试你的 DDP 类，首先实现 adapters `[adapters.get_ddp]` 和 `[adapters.ddp_on_after_backward]`（后者是可选的，根据你的实现可能不需要）。

然后运行测试：`uv run pytest tests/test_ddp.py`。我们建议多次运行测试（例如 5 次），以确保它能可靠通过。

Problem (`ddp_overlap_individual_parameters_benchmarking`)：重叠单个参数通信的 DDP 基准测试（1 分）

(a) 当 backward pass 计算与单个 parameter gradients 的通信重叠时，benchmark 你的 DDP 实现的性能。与之前研究的设置比较：最小 DDP 实现要么对每个 parameter tensor 发起一次 all-reduce，要么对所有 parameter tensors 的 concatenation 发起一次 all-reduce。使用相同设置：1 node、2 GPUs，以及 Section 2.1.2 中描述的 `xl` 模型尺寸。

Deliverable：当 backward pass 与单个 parameter gradients 的通信重叠时，每次 training iteration 的测得时间，以及 1-2 句话比较结果。

(b) 使用 Nsight profiler 对 benchmark 代码进行 instrument（使用 1 node、2 GPUs、`xl` 模型尺寸设置），比较初始 DDP 实现与该 overlapped 实现。视觉比较两个 traces，并提供 profiler 截图，展示一个实现将 compute 与 communication 重叠，而另一个没有。

Deliverable：2 张截图（一张来自初始 DDP 实现，另一张来自这个将 compute 与 communication 重叠的 DDP 实现），视觉展示 communication 是否与 backward pass 重叠。

6 优化器状态分片

Distributed data parallel training 在概念上很简单，并且通常非常有效，但它要求每个 rank 持有一份模型参数和 optimizer state 的独立副本。这种冗余可能带来显著内存成本。例如，AdamW optimizer 为每个参数维护两个 floats，这意味着它消耗的内存是模型 weights 的两倍。S. Rajbhandari et al. [5] 描述了几种方法，通过在 ranks 之间划分 (1) optimizer state、(2) gradients 和 (3) parameters，并在必要时在 workers 间通信它们，从而减少 data-parallel training 中的冗余。

在这一部分，我们将通过实现 optimizer state sharding 的简化版本来降低每个 rank 的内存消耗。不是为所有参数保留 optimizer states，而是让每个 rank 的 optimizer instance 只处理参数的一个子集（约为 `1 / world_size`）。当每个 rank 的 optimizer 执行

optimizer step 时，它只会更新其 shard 中的模型参数子集。然后，每个 rank 会把自己更新后的参数 broadcast 给其他 ranks，以确保每次 optimizer step 后模型参数保持同步。

Problem (`optimizer_state_sharding`)：优化器状态分片（15 分）

实现一个 Python 类来处理 optimizer state sharding。该类应当包装任意输入 PyTorch `optim.Optimizer`，并负责在每次 optimizer step 后同步更新后的参数。我们推荐以下 public interface：

`def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any)`：初始化 sharded state optimizer。`params` 是要优化的参数集合（或参数组，当用户想对模型不同部分使用不同超参数，例如不同 learning rates 时）；这些参数会跨所有 ranks 被 sharded。`optimizer_cls` 参数指定要包装的 optimizer 类型（例如 `optim.AdamW`）。最后，任何剩余 keyword arguments 都会转发给 `optimizer_cls` 的 constructor。确保在该方法中调用 `torch.optim.Optimizer` superclass constructor。

`def step(self, closure, **kwargs)`：使用提供的 `closure` 和 keyword arguments 调用被包装 optimizer 的 `step()` 方法。更新参数之后，与其他 ranks 同步。

`def add_param_group(self, param_group: dict[str, Any])`：该方法应当向 sharded optimizer 添加一个 parameter group。它会在 sharded optimizer 构造期间被 superclass constructor 调用，也可能在训练期间被调用（例如逐渐 unfreezing 模型中的 layers）。因此，该方法应当处理在 ranks 之间分配模型参数。

Deliverable：实现一个 container class 来处理 optimizer state sharding。为了测试你的 sharded optimizer，首先实现 adapter `[adapters.get_sharded_optimizer]`。然后运行测试：`uv run pytest tests/test_sharded_optimizer.py`。我们建议多次运行测试（例如 5 次），以确保它们可靠通过。

现在我们已经实现了 optimizer state sharding，接下来分析它对训练期间峰值内存使用和运行时开销的影响。

Problem (`optimizer_state_sharding_accounting`)：优化器状态分片核算（5 分）

(a) 创建一个脚本，profile 使用和不使用 optimizer state sharding 训练语言模型时的峰值内存使用。使用标准配置（1 node、2 GPUs、`xl` 模型尺寸），报告模型初始化后、optimizer step 之前，以及 optimizer step 之后的峰值内存使用。结果是否符合你的预期？分解每种设置中的内存使用（例如参数占多少内存、optimizer states 占多少内存等）。

Deliverable：2-3 句话的回答，包含峰值内存使用结果，并分解内存在不同模型和 optimizer 组件之间如何划分。

(b) 我们的 optimizer state sharding 实现如何影响训练速度？在标准配置（1 node、2 GPUs、`xl` 模型尺寸）下，测量使用和不使用 optimizer state sharding 时每次迭代所需时间。

Deliverable：2-3 句话，包含你的计时结果。

(c) 我们的 optimizer state sharding 方法与 ZeRO stage 1（在 S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He [5] 中称为 ZeRO-DP `P_os`）有何不同？

Deliverable：2-3 句话总结任何差异，尤其是与内存和通信量相关的差异。

7 全分片数据并行

有了 optimizer state sharding 和 data parallel，我们能够沿 data-parallel 轴拆分 optimizer state 和 activations。然而，模型 weights 仍然是重复的：我们在每块 GPU 上都存储了完整副本。

我们可以通过把 data parallel（DP）轴变成 fully-sharded data parallel 轴（FSDP）来解决这个问题。在 FSDP 中，每块 GPU 只存储每个 weight tensor 的自身切片，但必须从其他 GPUs 拉取切片，通过 all-gather 形成完整 weight tensor，以准备 forward 或 backward pass。

为了避免 GPU compute 等待 communication 完成，大多数 FSDP 实现会在操作之前提前调度该 layer 的 all-gather，这意味着相关 weights 在需要之前就已经准备好，防止 communication 阻塞 computation。这使得 weight sharding communication 不在 critical path 上，也就是说，只要 communication 能跟上 compute 且调度良好，它就没有成本。

有些 layers 在内存和计算上都足够小，以至于一次传输的 latency overhead 并不值得。你应当将这些 layers 标记为不被 FSDP sharded。在我们的架构中，这主要会是 norms。这使我们剩下 embedding layer 和每个 linear layer。

虽然必须以 FP32 存储 master weights（任何会被反复累加的值都对精度敏感），但 weights 不需要以 FP32 被使用。在 mixed precision 中，我们总是在使用之前转换为低精度 compute datatype，因此不妨在通信之前就转换，以节省带宽。

Problem (`fsdp`)：Fully-Sharded Data Parallel（15 分）

实现一个用于 fully-sharded data parallel training 的 Python 类。该类应当包装任意 PyTorch `nn.Module`（你的完整模型），并 hook into 或包装其中任何 Linear 或 Embedding layer。我们推荐以下 public interface：

`def __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None)`：给定一个要并行化的已实例化 PyTorch `nn.Module`，构造一个 FSDP module，它会处理 weight all-gathers 和 gradient reduce-scatters。确保你的 hooks 或 module wrappers 会及时为 forward pass all-gather weights。为了限制内存使用，仅在当前 layer 前两个 layer 已完成其 forward pass 后才开始 gather。在 backward pass 中，你的 hooks 或 module wrappers 应 all-gather 以便 weights 可用于计算。当 gradients 可用时，它们应 reduce-scatter 到合适的 ranks。确保使用后释放 gathered weights。当提供 `compute_dtype` 时，在通信或计算使用之前把 weights cast 为该 dtype，同时保持 master weights 和 optimizer updates 为 FP32。

`def forward(self, *inputs, **kwargs)`：使用提供的位置参数和关键字参数调用被包装 module 的 `forward()` 方法。

`def finish_gradient_synchronization(self)`：调用时，等待异步通信调用在 GPU 上完成。

Deliverable：实现一个 container class 来处理 fully sharded data parallel training。该 container 的每个 shard 应与作业 1 中的标准 AdamW 实现兼容。为了测试你的 FSDP 实现，请实现 adapter `[adapters.get_fsdp]`。使用 `uv run pytest tests/test_fsdp.py` 运行测试。我们建议多次运行测试（例如 5 次）以捕获任何 race conditions。

Problem (`fsdp_accounting`): FSDP 核算（5 分）

(a) 根据你在 Section 6 中的分析，你预计实现 FSDP 后能从峰值内存中节省多少内存？在计算中可以忽略 all-gather weights 到每块 GPU 所需的预分配 buffers 大小。

Deliverable：2-3 句话的回答，包含你的发现。

(b) 在两块 GPU 上 profile `xl` 模型，并关注 weights 的 all-gather。communication 是否能及时在 forward pass 前完成？

Deliverable：2-3 句话的回答，包含你的计时。包括 Nsight 截图来支持你的论断。

8 分析并行策略

我们还可以沿更多轴来并行化训练过程。一些常见策略包括：

- Data parallelism（DP）：data batches 被拆分到多个 devices 上，每个 device 为自己的 batch 计算 gradients。随后，gradients 会跨 devices 取平均。
- Fully-Sharded Data Parallelism（FSDP）：在 data parallelism 的基础上，还将 optimizer states、gradients 和 weights 跨 devices 拆分，以降低内存使用。随后，devices 需要在 forward 和 backward passes 期间从其他 devices gather weight shards。
- Tensor Parallelism（TP）：weight matrices 沿输入或输出维度 sharded。devices 计算与自身 shard 对应的 activations，然后 activations 会跨 devices reduce 或 gather。
- Pipeline Parallelism（PP）：模型按 layer 切分为多个 stages，每个 stage 在不同 device 上运行。
- Expert Parallelism（EP）：Mixture-of-Experts 模型中的 experts 被拆分到不同 devices 上，每个 device 计算自身 expert 的输出。

在本节中，我们将在简化设置下做一些基本数学，用于在并行策略之间做选择，并决定如何组合它们。首先，我们将关注 DP、FSDP、TP 及其组合。我们的方法是计算每种策略的通信成本，并将其与计算成本比较，这告诉我们在通信成本成为瓶颈之前可以扩展到多少 devices。

关于 TPU/GPU 拓扑和并行策略的更详细讨论，TPU Scaling Book（J. Austin et al. [6]）是一个很好的资源。关于更详细的 pipeline parallel 讨论，请参见 The Ultra-Scale Playbook Appendix（H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom [7]）。这些书的其余部分也有许多你可能会觉得有用的信息。

8.1 通信原语

第一步是理解通信原语。在简化设置中，假设我们有 `N` 个 devices，编号为 `0, ..., N - 1`，每对 devices 之间都有一条 link。还假设每个 device 具有 `W` egress（即 outgoing）bandwidth；换句话说，每个 device 可以以每秒 `W` bytes 的速率向另一个 device 发送数据。我们可以如何实现 gather 和 reduce？

实现 all-gather 操作的一种常见方式是 ring all-gather。回忆 all-gather 中，每个 device `i` 一开始有一个大小为 `S/N` 的 chunk `x_i`，最终得到大小为 `S`（以 bytes 计）的整个 `x = [x_0, ..., x_{N-1}]`。在 ring all-gather 中，我们把 devices 排列成一个环。在每一步中，每个 device 都把当前 chunk 发送给右侧的下一个 device，并存储从左侧 device 收到的 chunk。这个过程重复进行：每个 device 把刚收到的 chunk 向右传递，并从左侧接收一个新 chunk。经过 `N - 1` 步后，每个 device 都拥有完整 tensor。

在我们的理想化设置中，每个 device 在每一步同时传输大小为 `S/N` 的 chunk，egress bandwidth 为 `W`，共有 `N - 1` 步，因此 ring all-gather 需要 `(N - 1)/N * S/W` 秒。

接下来，分析 ring reduce-scatter。在 reduce-scatter 中，每个 device `i` 一开始有一个大小为 `S` 的完整 tensor `x^(i)`。随后我们希望计算 reduction `y = sum_{i=0}^{N-1} x^(i)`，但每个 device `i` 最终只得到大小为 `S/N` 的一个 chunk `y_i`。与 ring all-gather 一样，我们先把 devices 排列成一个环。每个 device 首先把自己的 tensor `x^(i)` 划分为 `N` 个 chunks `[x_0^(i), ..., x_{N-1}^(i)]`，每个大小为 `S/N`。然后我们像 ring all-gather 一样传递 chunks，只不过在传递 chunk 之前，每个 device 会把自身贡献加到该 chunk（它存储了一个 partial sum）上。具体来说：

对于步骤 `t = 1, ..., N - 1`，device `i` 执行：

- 如果 `t = 1`，初始化 `y <- x^(i)`，它存储到目前为止的 partial sum。
- 将 chunk `y_{(i-t) mod N}` 发送给 device `(i + 1) mod N`。
- 从 device `(i - 1) mod N` 接收 chunk `z_{(i-t-1) mod N}`。
- 更新你的 partial sum 副本：`y_{(i-t-1) mod N} <- y_{(i-t-1) mod N} + z_{(i-t-1) mod N}`。

经过 `N - 1` 步后，device `i` 拥有 chunk `y_i` 的完整 sum，因此 ring reduce-scatter 与 ring all-gather 一样，需要 `(N - 1)/N * S/W` 秒。

最后，实现 ring all-reduce。在 all-reduce 中，每个 device `i` 一开始有一个大小为 `S` 的完整 tensor `x^(i)`，最终得到 reduction `y = sum_{i=0}^{N-1} x^(i)`。我们将 all-reduce 实现为 ring reduce-scatter 后接 ring all-gather，因此 ring all-reduce 需要 `2 * (N - 1)/N * S/W` 秒。

Problem (`alternate_ring_all_reduce`)：另一种 ring all-reduce（1 分）

不是把 all-reduce 实现为 ring reduce-scatter 后接 ring all-gather，而是使用以下算法：

对于步骤 `t = 1, ..., N - 1`，device `i` 执行：

- 如果 `t = 1`，初始化 `y <- x^(i)`，它存储到目前为止的 partial sum。
- 将 `x^((i-t+1) mod N)` 发送给 device `(i + 1) mod N`。
- 从 device `(i - 1) mod N` 接收 `x^((i-t) mod N)`。
- 更新你的 partial sum 副本：`y <- y + x^((i-t) mod N)`。

在与上面相同的设置中（每个 device 的 egress bandwidth 为 `W`，每个 `x^(i)` 的大小为 `S`），该算法需要多久？

Deliverable：用 `S`、`N` 和 `W` 表示的答案，以及一句话说明理由。

8.2 分析数据并行

有了通信原语后，我们就可以分析并行策略。我们将分析单个 FFN layer 的并行化。回忆给定输入 `x` 时，forward pass 为：

```
x1 = x W1                                   (20)
x2 = x W2                                   (21)
z = f(x1) * x2                              (22)
y = z W3                                    (23)
```

其中 `x` 的形状为 `(B, D)`，`W1` 和 `W2` 的形状为 `(D, D_FF)`，`W3` 的形状为 `(D_FF, D)`。`f` 是 elementwise activation function（例如 SiLU），`*` 表示 elementwise multiplication。

显式写出 backward pass 也很有用。回忆给定形状为 `(B, D)` 的 `dy` 时，backward pass 为：

```
dz = dy W3^T                                (24)
dx2 = dz * f(x1)                            (25)
dx1 = dz * f'(x1) * x2                      (26)
dx = dx1 W1^T + dx2 W2^T                    (27)
dW3 = z^T dy                                (28)
dW2 = x^T dx2                               (29)
dW1 = x^T dx1                               (30)
```

其中 `*` 表示 elementwise multiplication。

回忆在使用 `N_DP` 个 devices 的 data parallelism 中，我们把输入 `x` 切分为 shards `x^(i)`，大小为 `(B / N_DP, D)`。forward pass 像往常一样进行，不需要任何 collectives，并产生大小为 `(B / N_DP, D)` 的 activations `y^(i)`。在 backward pass 中，继续照常使用 batch-sharded activations，device `i` 最终得到 gradients：

```
dW3^(i) = (z^(i))^T dy^(i)                  (31)
dW2^(i) = (x^(i))^T dx2^(i)                 (32)
dW1^(i) = (x^(i))^T dx1^(i)                 (33)
```

这里不是对所有 `B` 个 outer products 求和，而只是对输入 shard 中的 `B / N_DP` 个 outer products 求 partial sum。随后，我们需要跨 devices 进行 all-reduce，以得到完整 gradients `dW3, dW2, dW1`。

Problem (`data_parallel_calcs`)：Data parallel 计算（3 分）

现在我们具备了计算 data parallelism 何时成为通信瓶颈所需的一切。令 `C`（单位 FLOP/s）表示 device accelerator speed，`W`（单位 bytes per second）表示每个 device 的 egress bandwidth。随后我们可以计算 computation time 和 communication time。由于 computation 和 communication 可以重叠，当 communication time 大于 computation time 时，我们就会被瓶颈化。假设所有 weights 和 activations 都使用 FP16（即两个 bytes）。

(a) 使用 N_DP data parallelism 时，计算 backward pass 需要多少 FLOPs? 可以忽略所有非 matmul 操作。回忆 matmul `(A, B)(B, C) -> (A, C)` 需要 `2ABC` flops。

Deliverable：一个用 `B, D, D_FF, N_DP` 表示的答案，以及一句话说明理由。

(b) 使用 N_DP data parallelism 时，backward pass 需要多少 communication time?

Deliverable：一个用 `B, D, D_FF, N_DP, W` 的子集表示的答案，以及一句话说明理由。

(c) 固定其他参数时，在我们成为 communication bottlenecked 之前，N_DP 可以变得多大?

Deliverable：一个不等式，其中一边为 N_DP，另一边是用 `B, D, D_FF, C, W` 的子集表示的表达式，并附一句话说明理由。

8.3 分析全分片数据并行

接下来，分析 FSDP。回忆与 DP 一样，FSDP 会对输入和 activations 的 batch dimension 进行 sharding。此外，为节省内存，我们还会 shard optimizer states、gradients 和 weights。

我们可以沿任一维度 shard weights，在 device `i` 上产生 shards `W1^(i), W2^(i), W3^(i)`，每个大小为 `D D_FF / N_FSDP`。

在 forward pass 中，我们只是在 batch-sharded input 上做 data parallel forward pass。但要这样做，回忆我们需要先跨 devices all-gather weight shards:

```
W1 = all-gather({W1^(i)}_{i=0}^{N_FSDP-1})      (34)
W2 = all-gather({W2^(i)}_{i=0}^{N_FSDP-1})      (35)
W3 = all-gather({W3^(i)}_{i=0}^{N_FSDP-1})      (36)
(do batch-sharded forward pass)                 (37)
```

请注意，我们在什么时候做 all-gather 有一些自由度：只需要在 weight 被使用之前有 all-gather 后的 weight，并且随后应丢弃它以保持内存成本较低。为简单起见，本节只把三次 all-gathers 放在一起列出。

与 forward 一样，在 backward pass 中，我们需要首先跨 devices all-gather weight shards。随后可以做 data parallel backward pass，只不过不再需要 all-reduce，因为每个 device 只需要自身 shard 的 gradient。因此，我们做 reduce-scatter:

```
W1 = all-gather({W1^(i)}_{i=0}^{N_FSDP-1})      (38)
W2 = all-gather({W2^(i)}_{i=0}^{N_FSDP-1})      (39)
W3 = all-gather({W3^(i)}_{i=0}^{N_FSDP-1})      (40)
(do batch-sharded backward pass)                (41)
dW1^(i) = reduce-scatter({dW1^(i)}_{i=0}^{N_FSDP-1})    (42)
dW2^(i) = reduce-scatter({dW2^(i)}_{i=0}^{N_FSDP-1})    (43)
dW3^(i) = reduce-scatter({dW3^(i)}_{i=0}^{N_FSDP-1})    (44)
```

注意 sharding 记号（上标 `(i)`）被重载了：reduce-scatter 的输入是完整 weights 上的 partial sums，而输出是 sharded weights 上的 full sums。

Problem (`fsdp_calcs`): Fully sharded data parallel 计算（3 分）

在与 data parallel calculations 相同的设置下，让我们计算 FSDP 何时成为 communication bottlenecked。

(a) 使用 N_FSDP FSDP 时，计算 backward pass 需要多少 FLOPs? forward pass 呢?

Deliverable：两个用 `B, D, D_FF, N_FSDP` 表示的答案，以及两句一句话理由。

(b) 使用 N_FSDP FSDP 时，backward pass 需要多少 communication time? forward pass 呢?

Deliverable：两个用 `B, D, D_FF, N_FSDP, W` 的子集表示的答案，以及两句一句话理由。

(c) 固定其他参数时，在 backward pass 成为 communication bottlenecked 之前，N_FSDP 可以变得多大? forward pass 呢?

Deliverable：两个不等式，每个不等式一边为 N_FSDP，另一边是用 `B, D, D_FF, C, W` 的子集表示的表达式，并附两句一句话理由。

8.4 分析张量并行

在实践中，FSDP 常常与一种称为 tensor parallelism（TP）的并行策略组合。在 TP 中，我们会沿每个 weight matrix 的输入或输出维度进行 sharding。输入维度 sharding 常称为 “row parallel”，而输出维度 sharding 常称为 “column parallel”。

具体而言，假设我们想 shard matmul `xW`，其中 `x` 的形状为 `(B, D)`，`W` 的形状为 `(D, D_FF)`。在 column parallel 中，我们有形状为 `(D, D_FF / N_TP)` 的 shards `W^(i)`，并且:

```
xW = all-gather({x W^(i)}_{i=0}^{N_TP-1})       (45)
```

另一方面，对于 row parallel，我们有形状为 `(D / N_TP, D_FF)` 的 shards `W^(i)`，并且每个 device 在做 matmul 之前也把输入 `x` 缩窄为一个形状为 `(B, D / N_TP)` 的 slice `x^(i)`。然后有：

$$xW = \text{all-reduce}(\{x^{(i)}\ W^{(i)}\}_{i=0}^{N_{TP}-1}) \quad (46)$$

为了并行化我们的 FFN，我们将使用一个特定 tensor parallel 配置：`W1` 和 `W2` 是 column parallel（output-dimension-sharded），而 `W3` 是 row parallel（input-dimension-sharded）。由于 row parallel weight 只需要输入的一个 slice，这一配置让我们可以跳过 column parallel weights 之后的 all-gather。给定大小为 `(B, D)` 的输入 `x`，该策略的 forward pass 如下：

$$x1^{(i)} = x\ W1^{(i)} \quad (47)$$
$$x2^{(i)} = x\ W2^{(i)} \quad (48)$$
$$z^{(i)} = f(x1^{(i)}) * x2^{(i)} \quad (49)$$
$$y^{(i)} = z^{(i)}\ W3^{(i)} \quad (50)$$
$$y = \text{all-reduce}(\{y^{(i)}\}_{i=0}^{N_{TP}-1}) \quad (51)$$

其中 `W1^(i)` 和 `W2^(i)` 的形状为 `(D, D_FF / N_TP)`，`W3^(i)` 的形状为 `(D_FF / N_TP, D)`。

Problem (`tp_calcs`): Tensor parallel 计算（4 分）

在与 DP 和 FSDP calculations 相同的设置下，让我们计算 TP 何时成为 communication bottlenecked。

(a) 给定大小为 `(B, D)` 的输入 `dy`，写出上述 tensor parallel 策略的 backward pass（其中 `W1^(i)` 和 `W2^(i)` 的形状为 `(D, D_FF / N_TP)`，`W3^(i)` 的形状为 `(D_FF / N_TP, D)`）。

Deliverable：一系列描述 backward pass 的方程，使用 `dy`、sharded weights (`W1^(i), W2^(i), W3^(i)`)、从 forward pass 保存的 activations (`x, x1^(i), x2^(i), z^(i), y^(i)`)、communication primitives，以及你想定义的任何中间变量。方程应产生每个 device 的 gradients `dW1^(i), dW2^(i), dW3^(i)`，以及 backward pass 输出 `dx`。你可以参考 Section 8.2 中的 non-sharded backward pass 并加以修改。

(b) 使用 `N_TP` TP 时，计算 forward pass 需要多少 FLOPs? backward pass 呢?

Deliverable：两个用 `B, D, D_FF, N_TP` 表示的答案，以及两句一句话理由。

(c) 使用 `N_TP` TP 时，forward pass 需要多少 communication time? backward pass 呢?

Deliverable：两个用 `B, D, D_FF, N_TP, W` 的子集表示的答案，以及两句一句话理由。

(d) 固定其他参数时，在 backward pass 成为 communication bottlenecked 之前，`N_TP` 可以变得多大? forward pass 呢?

Deliverable：两个不等式，每个不等式一边为 `N_TP`，另一边是用 `B, D, D_FF, C, W` 的子集表示的表达式，并附两句一句话理由。

8.5 2D 并行（FSDP + TP）

我们终于可以组合并行策略了！在本节中，我们将研究如何最优地组合 FSDP 和 TP。提示：在前几节中，你应当已经发现 batch size 和 model size 参数会限制你能扩展到多少 devices；让所有东西变大可以让你继续扩展 devices 而不受 communication bottleneck 限制。不幸的是，batch size 扩大到某个点之后会开始降低性能，因为 gradient noise 显著缩小，从而失去 SGD 的隐式正则化性质；这个点常称为 "critical batch size"。而 scaling laws 往往告诉我们希望模型有多大（这将是你下一次作业中的任务！）。

在本节中，我们考虑一个简化设置：有人把所有问题参数（batch size、model size、bandwidth、accelerator speed）都给你。你的任务是选择 FSDP 和 TP 的配置，使其能够扩展到尽可能多的 devices，同时保持 compute-bound 而不是 communication-bound。

首先来走一遍把 FSDP 与 TP 组合的机制。每个 device 会有一个 TP rank `i = 0, ..., N_TP - 1` 和一个 FSDP rank `j = 0, ..., N_FSDP - 1`，形成一个总设备数为 `N = N_TP N_FSDP` 的 2D grid。按照 TP，我们首先沿输出维度 shard `W1` 和 `W2`，沿输入维度 shard `W3`。因此，我们必须在 activations 上插入 TP 风格的 all-reduce。接着应用 FSDP，我们会 shard inputs 的 batch dimension，并进一步沿 TP 未 shard 的维度 shard 每个 weight matrix。然后，我们必须在执行 TP 风格 forward/backward passes 之前，在 weights 上插入 FSDP 风格的 all-gathers，并在 TP 风格 backward pass 之后，在 weight gradients 上插入 reduce-scatters。

结果是，每个 device `(i, j)` 持有 weight shards `W1^(i,j)` 和 `W2^(i,j)`，形状为 `(D / N_FSDP, D_FF / N_TP)`，以及 `W3^(i,j)`，形状为 `(D_FF / N_TP, D / N_FSDP)`。随后，给定大小为 `(B / N_FSDP, D)` 的 batch-sharded input `x^(j)`，forward pass 可以写为：

$$W1^{(i)} = \text{all-gather}(\{W1^{(i,j)}\}_{j=0}^{N_{FSDP}-1}) \quad (52)$$
$$W2^{(i)} = \text{all-gather}(\{W2^{(i,j)}\}_{j=0}^{N_{FSDP}-1}) \quad (53)$$
$$W3^{(i)} = \text{all-gather}(\{W3^{(i,j)}\}_{j=0}^{N_{FSDP}-1}) \quad (54)$$
$$x1^{(i,j)} = x^{(j)}\ W1^{(i)} \quad (55)$$
$$x2^{(i,j)} = x^{(j)}\ W2^{(i)} \quad (56)$$
$$z^{(i,j)} = f(x1^{(i,j)}) * x2^{(i,j)} \quad (57)$$
$$y^{(i,j)} = z^{(i,j)}\ W3^{(i)} \quad (58)$$
$$y^{(j)} = \text{all-reduce}(\{y^{(i,j)}\}_{i=0}^{N_{TP}-1}) \quad (59)$$

最终得到大小为 `(B / N_FSDP, D)` 的 batch-sharded output `y^(j)`。为简洁起见，本节省略 backward pass，只关注 forward pass。但此时，你应当已经拥有自己写出它所需的所有信息。

Problem (`fsdp_tp_calcs`)：2D parallelism 计算（6 分）

在与目前计算相同的设置下，让我们计算 2D parallelism 何时成为瓶颈。

(a) 使用 `N_FSDP` FSDP + `N_TP` TP 时，计算 forward pass 需要多少 FLOPs?

Deliverable：一个用 `B, D, D_FF, N_FSDP, N_TP` 表示的答案，以及一句话说明理由。

(b) 使用 `N_FSDP` FSDP + `N_TP` TP 时，forward pass 需要多少 communication time? 假设沿每个轴的通信可以重叠（换句话说，沿 FSDP 轴的 collectives 可以与沿 TP 轴的 collectives 重叠）。

Deliverable：一个用 `B, D, D_FF, N_FSDP, N_TP, W` 的子集表示的答案，以及一句话说明理由。提示：答案应表示为两个量（FSDP 和 TP collective costs）的 max，因为二者可以重叠。

(c) 在 `N_TP` 和 `N_FSDP` 的最优设置下，在 forward pass 成为 communication bottlenecked 之前，`N = N_TP N_FSDP` 可以变得多大?

Deliverable：一个不等式，其中一边为 `N`，另一边是用 `B, D, D_FF, C, W` 的子集表示的表达式，并附几句话和方程作为理由。

(d) 现在假设 FSDP 轴和 TP 轴的 collectives 无法重叠，因为它们共享相同网络资源。在 `N_TP` 和 `N_FSDP` 的最优设置下，在 forward pass 成为 communication bottlenecked 之前，`N = N_TP N_FSDP` 可以变得多大? 不用担心把 `N_TP` 和 `N_FSDP` 截断为整数。

Deliverable：一个不等式，其中一边为 `N`，另一边是用 `B, D, D_FF, C, W` 的子集表示的表达式，并附几句话和方程作为理由。

9 排行榜

作业 2 的 leaderboard 将测试一个 8B 模型的完整 training step 速度。我们挑战你 benchmark 你的代码，并使用任何你能想到的技巧优化内存和运行时间。关键限制是你不能改变模型的输入/输出行为。你的实现会针对 `cs336_basics` 目录中的模型进行测试。你的输入会在 BF16 和 causal masking 下测试，并且必须通过与你常规实现相同的测试。实现还必须是你自己的，不能使用或复制已有实现。你的计时应在两块 B200 GPUs 上测量，使用 batch size 2、sequence length 32,768 的样本。有意让该模型很难放入内存。完整配置见下面的代码。我们会验证前 5-10 个提交的正确性和性能。我们将用于计时你实现的测试如下:

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
    is_causal = True
    batch_size = 2
cfg = Config()

def test_timing_forward_backward():
    labels, targets = torch.randint(high=cfg.vocab_size, size=(2, cfg.batch_size,
cfg.ctx_len))

     model = BasicsTransformerLM(Config())
     optimizer = AdamW(model.parameters())

     def train_step():
         optimizer.zero_grad(set_to_none=True)
         res = model(labels)
         loss = cross_entropy(res, targets).sum()
         loss.backward()
         optimizer.step()

     timing_results = triton.testing.do_bench(train_step, rep=30_000, warmup=10_000)
     print(timing_results)
```

为了测试，你可以把 repetition 和 warmup time（以 ms 给出）减少到更短的值。

一些可能的改进和确保模型能放入内存的想法:

- 调优你的 kernel 的 tile sizes（为此使用 Triton autotune!）
- 调优额外的 Triton/`torch.compile` config 参数
- 实现 fused AdamW
- 基础实现会物化完整 logits（`[batch, seq_len, vocab_size]`）。编写一个 kernel，融合 LM head 和你的 cross-entropy loss。你也可以让它立即以融合方式计算 backward pass。
- 改进 FlashAttention
  - 在 Triton 中实现 backward pass，而不仅是 `torch.compile`（见 Section 4.2.3）。
  - 对 backward pass 的输入做两次 pass，一次用于 `dQ`，另一次用于 `dK` 和 `dV`，以避免 atomics 或 blocks 之间的 synchronization。
  - 进行 causal masking 时，让 program instances 提前停止，跳过保证全为零的 tiles。
  - 将非 masked tiles 与 tile diagonals 分开：前者完全不比较 indices，后者只做一次 comparison。
  - 在 Hopper 之后的架构上使用 TMA（Tensor Memory Accelerator）功能，遵循与教程类似的模式。
- 仅在需要时使用 activation checkpointing，以 runtime speed 换取 memory savings。

Problem (`leaderboard`)：Leaderboard：最快 training step（10 分）

该 benchmark 会在 batch size 2、两块 B200 GPUs 上运行。你的提交会按完整 training step 的 wall-clock time 评估：forward pass、loss、backward pass 和 AdamW update。

从空的 PyTorch/Triton cache 开始，你的 benchmarking run 必须在 10 分钟内完成，因此请小心过于激进的 `torch.compile` 和 Triton autotuning。

Deliverable：使用 AdamW 的完整 forward-and-backward training step 的最佳 wall-clock time。

我们期望 leaderboard submissions 能击败 10 秒的朴素 baseline。

在这里提交你的结果到 leaderboard：

`github.com/stanford-cs336/assignment2-systems-leaderboard`

参考文献

[1] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." [Online]. Available: `https://arxiv.org/abs/2307.08691`

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Advances in Neural Information Processing Systems, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: `https://openreview.net/forum?id=H4DqfPSibmx`

[3] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax." [Online]. Available: `https://arxiv.org/abs/1805.02867`

[4] H. He, "Making Deep Learning Go Brrrr From First Principles," 2022, [Online]. Available: `https://horace.io/brrr_intro.html`

[5] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." 2020.

[6] J. Austin et al., "How to Scale Your Model," 2025.

[7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, "The Ultra-Scale Playbook: Training LLMs on GPU Clusters." 2025.